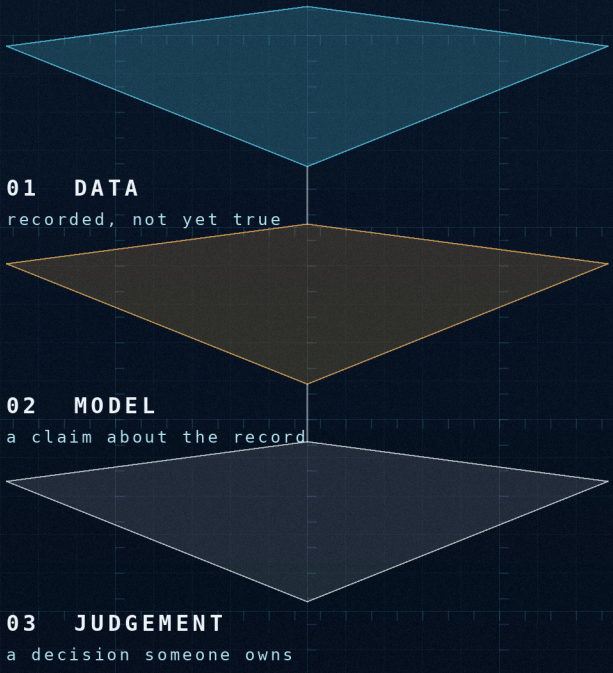
JACARANDA LABS PRESS · HARARE

Intelligence, Data and Judgement

An applied introduction to artificial intelligence, with a production case



COURAGE GARIKAI

FIRST EDITION · 2026

FIG. 1 — RAW RECORD, MODEL, AND THE PERSON WHO SIGNS

CONTENTS

About this textbook 4

Who should read it 6

How to read it 7

How to use it in teaching 8

A note on evidence and honesty 9

Part One — Foundations 10

1 What intelligence is, and why the question is old 11

2 Four definitions, one discipline 18

3 A short history that explains today's defaults 23

4 What AI can do, what it cannot, and when to use it 28

5 Intelligence in the small firm 33

Part Two — The Material 37

6 Data: the raw material 38

7 Preparing data: the lifecycle 44

8 Features: the representation that decides the outcome 50

9 Seeing data: descriptive statistics and visualisation 56

10 Learning from data 61

11 Measuring a model honestly 68

12 The arithmetic of being wrong 74

Part Three — The Machine 80

13 Models in production 81

14 Algorithms, procedures and pseudocode 87

15 Languages, APIs and the cloud 92

Part Four — The Case 98

16 Designing under constraint: the rail 99

17 The gate: finding prospects without lying to yourself . . 106

18 What broke 112

Part Five — Judgement 118

19 Ethics, consent and the things a system should refuse . 119

20 Law, data protection and the state 126

21 Teaching it: curriculum, sequence and assessment 133

22 Conclusion: the discipline of the second question 139

Appendix A · Glossary 143

Appendix B · Eight labs 148

Appendix C · Mapping to the three source curricula 150

Appendix D · References and sources 154

Colophon 159

About this textbook

This book teaches the fundamentals of artificial intelligence to people who will have to use it — engineers, operators, teachers, and the owners of small businesses in places where the technology arrives later than the marketing. It is written to be taught from: every chapter opens with learning outcomes, carries a worked example, and closes with exercises and further reading.

The book has a companion that most introductions do not: a running case. **Tapstall** is a merchant commerce rail that I built and operate in Zimbabwe, South Africa, Kenya and Nigeria. It runs on free infrastructure, holds no customer money, and uses AI in precisely the places where a small firm needs it — finding prospects, writing copy, reconciling orders. Every abstract idea in this book is attached to something Tapstall does, or to something Tapstall got wrong. Chapter Sixteen onwards handles the case directly; earlier chapters refer to it in passing, so that the theory never floats.

Three existing curricula form the disciplinary spine: the COMCEC introduction to AI and its fundamental concepts (Kurt, 2025), prepared under the Organisation of Islamic Cooperation's Standing Committee for Economic and Commercial Cooperation; Microsoft's Fundamentals of Artificial Intelligence, Module 1 (Microsoft, 2020); and Let's Learn Artificial Intelligence, the Base Module published by NITI Aayog's Atal Innovation Mission with NASSCOM (NITI Aayog, 2020). These are cited throughout as [COMCEC], [Microsoft] and [NITI Aayog], and Appendix D maps every chapter of this book onto the sections of all three, so

that an instructor teaching from any of them can slot this material in.

Who should read it

Undergraduate and postgraduate students taking a first or second course in AI who want the standard material connected to an actual deployment rather than a toy dataset.

Practitioners who can call an API but cannot explain why the model's answer should be trusted, and what to do when it is not.

Non-specialist professionals — accountants, teachers, operators, public servants — who need enough vocabulary and judgement to evaluate what a vendor is selling them.

Instructors who want a one-semester course with labs, exercises, a case, and a mapping to the three widely used curricula above.

How to read it

The book is in five parts. **Part One** asks what intelligence is, why the definitions differ, and how those differences survive into systems. **Part Two** is the material: data, features, statistics, learning, and the arithmetic of being wrong. **Part Three** is the machine: algorithms, languages, APIs, cloud economics, and the lifecycle of a model in production. **Part Four** is the case: three chapters of Tapstall, ending with a forensic account of what fails. **Part Five** is judgement: ethics, law, and pedagogy.

Part One and Part Two are the core of any course. Part Three can be compressed for a non-technical cohort. Part Four should not be skipped by anyone who intends to deploy anything, because it is where the book stops describing AI and starts describing what happens when AI meets a Tuesday.

How to use it in teaching

A twelve-week semester at one two-hour session per week fits chapters one to twenty with two weeks on Part Four. A shorter six-week module should take chapters one, two, four, six, eight, nine, ten, nineteen. Appendix B supplies eight labs — four of them runnable with no code at all — and Appendix C collects the exercises by chapter. Each chapter’s exercises are graded by difficulty: recall questions test vocabulary, applied questions require a calculation or a design, and critical questions have no single right answer and should be argued in class.

A note on evidence and honesty

Where this book states something about the three source curricula, it is quoting or paraphrasing their content, attributed as [COMCEC], [Microsoft] or [NITI Aayog]. Where it states something about Tapstall, the figure comes from the running system or its public history — the release chain, the test suite, the production endpoints — and I say when a figure is a design ceiling rather than an observation. Where the book argues a position rather than reporting a fact, it says so. This distinction is the book's own first lesson: the difference between a claim about the world and a claim about the author is the difference between data and opinion, and most failures of applied AI begin with a blurring of the two.

One further admission. Tapstall has no paying customers at the time of writing. It is a working system with a verified test surface and a forensic record, not a validated commercial success. The case is used in this book for its architecture and its failure modes, both of which are real and inspectable. Any instructor who wants a case about market success will need a different case; any instructor who wants a case about what actually goes wrong will find this one useful.

PART ONE

Foundations

CHAPTER 1

What intelligence is, and why the question is old

Learning outcomes. By the end of this chapter you should be able to: state three common criteria by which human intelligence is described; explain why a single definition of artificial intelligence is unavailable; distinguish narrow from general intelligence; and describe the Turing test and the Chinese Room experiment and the different questions each raises.

1.1 The question a student asks first

The first question in every AI course is the one with no settled answer: what is intelligence? NITI Aayog's Base Module opens Unit 1 with it directly, and its method is instructive — rather than offering a definition and defending it, the module offers a checklist. When are we called intelligent? When we answer questions correctly; when we solve a difficult puzzle; when we decide the right action; when we learn quickly; when we recognise things and remember their names; when we use clues to guess. All of these, the module concludes, are examples of natural human intelligence, and a common set of abilities runs through the definitions: the ability to observe, recognise and understand; the ability to make smart decisions and solve problems; and the ability to learn and increase knowledge. An agent that can do those things is, in the module's phrase, an intelligent agent [NITI Aayog].

That checklist is doing serious work, and a student should notice what kind of work it is. It converts a metaphysical

question — what is intelligence, really? — into an operational one: which abilities, exhibited by what kind of agent, will we count? Every definition of artificial intelligence in the literature is an answer to that operational question, and each answer chooses a slightly different checklist. This is why the field has never settled on one definition, and why the absence of a definition is not a scandal but a consequence of the question.

1.2 Five definitions and what each one buys

The literature offers definitions that cluster into families. Winston's is about computation: "the study of the computations that make it possible to perceive, reason, and act" (Winston, 1992, quoted in [COMCEC]). Kurzweil's is about capability relative to people: "the art of creating machines that perform functions that require intelligence when performed by people" (Kurzweil, 1990, quoted in [COMCEC]). Rich and Knight put it comparatively: "the study of how to make computers do things at which, at the moment, people are better" (Rich and Knight, 1991, quoted in [COMCEC]). Poole and colleagues frame it as agents: "computational intelligence is the study of the design of intelligent agents" (Poole et al., 1998, quoted in [COMCEC]). Nilsson's is the tersest: AI "is concerned with intelligent behavior in artifacts" (Nilsson, 1998, quoted in [COMCEC]).

Microsoft's module gives McCarthy's definition, and it is worth memorising because it is the one most often misquoted: AI is "the science and engineering of making intelligent machines, especially intelligent computer programs" (McCarthy, quoted in [Microsoft]). The phrase science and engineering matters. It places AI among disciplines that

build things and study them, rather than among disciplines that only study.

Each definition buys something and costs something. Kurzweil's and Rich and Knight's definitions are relative to human performance, which gives a measurable standard — if a machine does what a person does, we are done — but makes the field's targets move whenever people get better at something, and says nothing about how the machine does it. Winston's and Poole's definitions are about mechanism, which tells you where to look for progress, but leaves the boundary of the field fuzzy, since any well-designed agent qualifies. McCarthy's is a project description, broad enough to include work that has not yet produced intelligent behaviour at all.

For a student, the practical consequence is this: **when someone says "AI", ask which definition they are using, because their claims are only as strong as that choice.** A vendor who says "our AI understands your customers" is using a capability definition and implying a mechanism; the two are not the same, and the gap between them is where disappointment lives.

1.3 Narrow and general

The COMCEC material draws the distinction that most courses begin with. Narrow AI is effective only in the specific areas it was trained for; it deals with a specific system and a specific problem, and the examples given are fraud detection, facial recognition and social recommendations. General AI, which does not yet exist, is expected to function like human intelligence; the module names the central

difficulty as modelling the world consistently and usefully across an entire body of knowledge [COMCEC].

The distinction is often taught as a ladder with narrow at the bottom and general at the top, and that metaphor misleads in one important way: it suggests that narrow systems are partial versions of general ones, as a child is a partial version of an adult. They are not. A narrow system is a tool shaped to a task, and its narrowness is not a deficiency but the condition of its usefulness. A fraud detector that also had opinions about poetry would be worse at detecting fraud.

Where the distinction earns its keep is in the claims made around systems. Almost every deployment failure I have seen in small business software begins with someone treating a narrow system as though its competence extended one step beyond its training. The discovery system in Chapter Seventeen was narrow — it could find businesses in a directory and read a page for a phone number — and it failed precisely at the boundary, when it encountered a page that mentioned the right words without being a business.

1.4 Two thought experiments, two different questions

The COMCEC material presents the two canonical thought experiments, and students routinely conflate them because both involve a machine and a human in conversation.

The Turing test, proposed by Alan Turing in 1950, asks a judge to communicate in writing with a human and a machine and to determine which is which. If the judge cannot reliably distinguish them, the machine is said to have passed [COMCEC]. The test is behaviourist: it asks only

what the system does, not how. Its virtue is that it is operational and public — anyone can run it. Its vice is that it can be passed by imitation, and a system that imitates well may understand nothing.

Searle's Chinese Room, proposed in 1980, is a thought experiment about understanding. A person who does not know Chinese sits in a room and answers Chinese questions by following a rulebook, producing plausible answers without comprehension. People outside may conclude that the room understands Chinese; the person inside does not [COMCEC]. The argument's target is the claim that running the right program constitutes understanding.

Notice what each experiment is a test of. Turing's is a test of indistinguishability in behaviour. Searle's is an argument that indistinguishability in behaviour is not sufficient for understanding. The first is a research programme; the second is a philosophical objection to a claim about what that programme would achieve. Both remain useful in a classroom because they force the student to separate three questions that the word "intelligence" hides: can it do the thing? does it do it the way we do? and does it know what it is doing? Modern systems answer yes, no and no, respectively, and most public argument about AI is people answering different questions from each other.

1.5 A local history worth knowing

The COMCEC material includes a section on the Turkish mathematician Cahit Arf, who wrote in the late 1950s on whether machines could think and how they might [COMCEC]. It is a small inclusion with a large teaching purpose: it demonstrates that the question was never confined to

Cambridge, Massachusetts, and that contributions to the field's foundations came from many places.

This matters for students in Africa, Asia and Latin America, and not for reasons of sentiment. A field whose history is taught as though it happened in one country produces practitioners who assume that its defaults — its languages, its data sources, its regulatory assumptions, its idea of a normal user — are natural rather than local. They are local. Chapter Sixteen is about building for a market where the normal user has a two-year-old Android phone, an intermittent connection, and no reason to trust a company she has never heard of. Recognising that the field's defaults were chosen somewhere is the first step to choosing differently.

1.6 Summary

Intelligence is defined operationally in every curriculum, with a checklist of abilities rather than a single criterion. Definitions differ in what they measure, and claims inherit the weakness of the definition behind them. Narrow systems are tools, not incomplete people. The Turing test asks whether behaviour is distinguishable; the Chinese Room asks whether behaviour implies understanding. Both questions remain live, and the discipline of keeping them apart is the beginning of judgement.

Worked example — reading a claim. A company advertises: "Our AI reads your customers' messages and understands what they want." Which definition is being used, and what would you ask to test the claim? Answer: a capability definition (it performs a function humans perform) is being used to imply a mechanism (it comprehends intent). The testable version is narrower and specific: on a

sample of your messages, how often does the system's extracted intent match your own label, and what happens on the messages it gets wrong? That question can be answered with a spreadsheet and a morning's work. Chapter Ten is about turning exactly that question into arithmetic.

Exercises. (recall) List the three abilities NITI Aayog's module treats as common to definitions of human intelligence. (applied) Take any three AI definitions quoted in this chapter and write, for each, one claim a vendor could make under that definition that would be unwarranted under the other two. (critical) "The Turing test measures the wrong thing." Argue for or against, using the Chinese Room to sharpen your position.

Further reading. Turing (1950); Searle (1980); Russell and Norvig (2021), chapter 1; [NITI Aayog], Unit 1; [COMCEC], "What is Artificial Intelligence?".

CHAPTER 2

Four definitions, one discipline

Learning outcomes. By the end of this chapter you should be able to: place a definition of AI on the thinking/acting and human/rational axes; explain what each quadrant optimises for; and identify which quadrant a given product or paper occupies.

2.1 The grid

The COMCEC material arranges definitions in a two-by-two grid that every AI course eventually presents: whether the system is judged by thought or action, and whether the standard is human performance or rational performance [COMCEC]. The four cells are thinking like a human, thinking rationally, acting like a human, and acting rationally.

	Human standard	Rational standard
Thought	Thinking like a human: modelling cognition itself	Thinking rationally: the laws of thought, logic
Action	Acting like a human: behaviour indistinguishable from a person	Acting rationally: doing the best thing given what is known

The grid's origin is Russell and Norvig's textbook, and its pedagogical value is that it makes arguments commensurable. Two researchers who appear to disagree about AI are frequently disagreeing about which cell they are working in, and neither is wrong.

2.2 What each quadrant costs

Thinking like a human requires a theory of human cognition, and every such theory is contested. The strength of this programme is that it takes the only known example of general intelligence seriously; the weakness is that it inherits psychology's disagreements and has historically produced systems that mimic surface behaviour (response times, error patterns) without producing capability.

Thinking rationally replaces psychology with logic. Its ideal is inference: given premises, derive conclusions. This is the oldest formal programme in the field, and it produced the logic-based systems of the 1970s and 1980s. Its difficulty is the one every student meets in week three: the world is uncertain, and classical logic handles uncertainty badly without elaborate machinery.

Acting like a human is the Turing tradition. It is measurable, which is a great virtue, and it is gameable, which is a great hazard. A system optimised to be indistinguishable from a person in conversation may learn to imitate the style of understanding — hedges, pleasantries, confident sentences — rather than its substance.

Acting rationally is the agent tradition, and it is where most deployed systems now live. The unit of analysis is the agent and its environment; the standard is whether the action chosen is good given the agent's information and objectives. This framing is congenial to engineering because it lets you define success without defining consciousness. It is also the framing that makes evaluation possible, because "good given what is known" can be measured against outcomes.

2.3 Why the rational-agent framing dominates production

Nearly every system in this book's case operates in the acting-rationally cell, and the reason is not that the other cells are uninteresting. It is that the rational-agent framing gives you a specification.

Consider a system that must find small businesses from public directory pages and decide which are worth a salesperson's morning. Under a "thinking like a human" standard you would have to define what a human salesperson thinks, which is both hard and unnecessary. Under an "acting rationally" standard the specification is compact: given a page, decide whether a person could open that page and start a conversation, and be able to say why. That specification can be tested, and it can fail in ways you can enumerate — which is precisely what Chapter Seventeen describes.

The cost of the rational framing is that it can make a system locally rational and globally foolish, because "given what is known" is doing enormous work. A system optimising lead volume given a weak admission rule will rationally fill a queue with page titles. Nothing in the local specification is violated. The failure is in what was specified.

2.4 The definition you use shapes the test you build

This is the practical lesson of the chapter, and it should be written on the first page of any project brief.

If you define the task as "classify these pages as businesses or not", you will build a test set of pages and measure accuracy. If you define the task as "find prospects a salesperson can call", you will build a test set of calls and

measure how many connect, and you will discover things the first test cannot see — that a page can be a business with an unusable phone number, or a directory entry for a business that closed, or a legitimate company that must never be cold-called because it is a charity. The definition, not the algorithm, determined which failures were visible.

Microsoft's module makes the same point from the product side when it frames AI as the automation of functions that require intelligence when a person performs them, and then lists the human faculties involved — reasoning, learning, problem solving, perception, linguistic intelligence [Microsoft]. That list is a set of test surfaces. A system that claims to read customer messages is claiming perception and linguistic intelligence; the honest evaluation asks about each separately.

2.5 Summary

The four-definition grid is not trivia; it is a diagnostic. Naming the quadrant tells you what a system is optimised for, what its evaluation will measure, and which failures it will be blind to. Most production systems act rationally, which is why their specifications — not their algorithms — are the usual cause of failure.

Worked example — placing a product on the grid. A speech-to-text service advertises "human-level transcription". Which cell, and what follows? Answer: acting like a human, on the human standard, with a claim about a specific capability. What follows is that the evaluation must be comparative against human transcribers, on a corpus representative of the actual use — accented speech, background noise, code-switching between English and Shona — and not on a clean benchmark corpus. A result on clean

read speech tells you nothing about a Harare shop counter.

Exercises. (recall) Draw the grid and label each cell with one definition from Chapter One. (applied) Take a system you use — a spam filter, a recommendation feed, a transcription service — and write its specification in the acting-rationally form: agent, environment, information, objective, and what "good" means. (critical) Argue that the acting-rationally framing is unsuitable for systems that make judgements about people.

Further reading. Russell and Norvig (2021), chapter 2; [COMCEC], "Thinking Like a Human" through "Acting Rationally"; Simon (1955) on bounded rationality.

CHAPTER 3

A short history that explains today's defaults

Learning outcomes. By the end of this chapter you should be able to: name the founding event of the field and its participants' ambition; explain the pattern of boom and retrenchment; and identify the three inputs whose cost collapse produced the current wave.

3.1 Dartmouth, 1956

The field dates itself from a summer research project at Dartmouth College in 1956. The COMCEC material records the plan: eleven mathematicians and others to work for a summer on the conjecture that "every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it" [COMCEC]. John McCarthy, who proposed the term artificial intelligence for the workshop, is quoted in Microsoft's module for the definition that survives: the science and engineering of making intelligent machines, especially intelligent computer programs [Microsoft].

A student should read that 1955 conjecture carefully, because it is still the field's charter and still unproven. It is not a claim that machines are intelligent. It is a claim that intelligence can be precisely described, and that once described, it can be simulated. The intervening seventy years have been, in large part, a long argument about whether that is true of the things people actually do — recognise a

face, understand a sentence, choose an action — or only of narrow slices of them.

The COMCEC material also records the institutionalisation that followed, including the laboratory established at the Massachusetts Institute of Technology [COMCEC]. Labs matter in this history, not as trivia but because they set the field's assumptions about what problems are interesting. The first generation chose problems that were hard for people and easy to formalise — chess, theorem proving, puzzles. The consequence was a field that was, for two decades, excellent at games and poor at perception.

3.2 Two winters, and what they taught

The pattern of the field's history is confidence, demonstration, disappointment, retrenchment, and renewal — twice before the current wave. The first retrenchment followed the discovery that machine translation and general problem solving were far harder than the early demonstrations suggested; the second followed the collapse of the commercial expert-systems market in the late 1980s, a collapse that Minsky and Papert's earlier critique of the perceptron had foreshadowed from the theory side.

Two lessons from those periods deserve to be taught, because both are live in 2026.

Lesson one: capability demonstrations are not deployments. A system that works in a laboratory on curated inputs has demonstrated a capability, not a product. The gap between the two is made of the things this book's later chapters enumerate: data quality, maintenance, cost, failure modes, and the human work of fitting the system into an existing process.

Lesson two: the field forgets its own failures faster than it forgets its successes. Symbolic approaches were pronounced dead, then returned inside systems that combine them with statistical methods; connectionism was pronounced dead in 1969 and returned in 1986 and again in 2012. A student who reads only the current hype will believe that the present architecture is the first attempt, and will therefore not anticipate the ways it can fail.

3.3 What changed in 2012

The COMCEC material notes the 2024 Nobel Prize in Physics awarded to John Hopfield and Geoffrey Hinton, whose work in the 1980s established foundational results for neural networks and a technique for training them [COMCEC]. That award is a useful marker because it recognises work done decades before the current wave. The wave itself had three proximate causes, and every AI course should name them explicitly.

Data. The internet produced labelled and unlabelled corpora at a scale no research lab could assemble. Microsoft's module devotes a section to public datasets precisely for this reason [Microsoft].

Compute. Graphics processors, designed for a different purpose, turned out to be efficient at the matrix arithmetic neural networks require. The cost of a floating-point operation fell by orders of magnitude.

Algorithms. A set of modest improvements — activation functions that train better, regularisation that generalises better, architectures that exploit structure — made larger networks trainable. The transformer architecture, introduced in 2017, is the clearest example: not a new idea

about intelligence, but a new way of arranging computation that scaled better than what preceded it.

The historical point is that none of these three is understanding. They are the conditions under which a statistical programme became enormously useful. A student who understands this is immunised against two opposite errors: believing the systems understand, and believing they are worthless.

3.4 Why this history is not decoration

In a business context, the history explains why today's systems have the shape they do. They are **statistical** because they were trained on data rather than written as rules. They are **general-purpose** because the same architecture serves many tasks. They are **brittle at boundaries** for the same reason they are competent in the middle: they interpolate what they have seen, and they do not know what they have not seen. And they are **cheap to call and expensive to trust**, because the cost of a prediction collapsed while the cost of verifying it did not.

That last asymmetry is the economic fact behind most of this book. If a prediction costs a fraction of a cent and a verification costs a person ten minutes, then the design question is never "can the model do this?" but "which predictions will a person actually check, and what happens to the ones they do not?" Tapstall's admission gate, its statement audit and its refusal to hold money are all answers to that question.

3.5 Summary

The field's charter is a claim that intelligence can be described precisely and therefore simulated; its history is an

argument about how far that is true. Two retrenchments taught that demonstrations are not deployments and that the field forgets failures faster than successes. The current wave rests on data, compute and algorithms — conditions for usefulness, not evidence of understanding.

Worked example — dating a claim. A colleague says: "Neural networks were invented in 2012." What is wrong and why does it matter? Answer: the perceptron dates from 1958 and the backpropagation-trained multilayer network from the mid-1980s; 2012 marks a demonstration at scale (the ImageNet result), not an invention. It matters because it locates the change in data and compute rather than in a conceptual breakthrough — and therefore predicts correctly that the same architecture will fail where data is thin.

Exercises. (recall) Name the three inputs whose cost collapse produced the current wave. (applied) For a task you know well, estimate how much labelled data would be required before a statistical system became useful, and what fraction of that corpus exists in a language you need. (critical) "The history of AI is a history of overpromising, so current claims should be discounted." Discuss what discount rate you would apply and why.

Further reading. McCarthy et al. (1955); Krizhevsky et al. (2012); Vaswani et al. (2017); [COMCEC], "The Birth of Artificial Intelligence" and "The Intelligence Revolution".

CHAPTER 4

What AI can do, what it cannot, and when to use it

Learning outcomes. By the end of this chapter you should be able to: state the capabilities and limits the source curricula identify; apply the five-question test for whether a problem is suitable for AI; and produce a written judgement on a proposed AI project.

4.1 The capability list, read carefully

Microsoft's module lists areas of application that have become familiar: natural language generation, speech recognition, virtual agents, conversational AI, and the five API families of vision, speech, language, search and decision [Microsoft]. NITI Aayog's Unit 1 covers applications of AI in everyday life and asks students to recognise them [NITI Aayog]. COMCEC asks directly what AI can and cannot do, and answers partly by listing things no current system does — fall in love, enjoy strawberries and cream, make someone fall in love with it, learn from experience in the way a person does, use words properly, be the subject of its own thought, have as much diversity of behaviour as a person, or do something really new [COMCEC].

That last list is more useful than the first, and for a structural reason. Capability lists age quickly — each year adds and removes items — while the limits list names a kind of limit. The things on COMCEC's list are not tasks that have not yet been automated. They are the things that make a person a person: affect, embodiment, originality, self-reference. A system can imitate the output of each, sometimes

convincingly; it cannot be the thing producing them, because there is nothing there for the output to be about.

Students find this unsatisfying, and they are right to. “It can imitate but not be” is a philosophical statement, and philosophy does not ship software. The engineering translation is more useful: **a system’s competence is bounded by the distribution it was trained on, and by the definition of the task it was given.** Everything in the limits list is, operationally, a situation outside that distribution, or a task whose definition has been narrowed until the hard part disappeared.

4.2 “AI is not a superhero”

The COMCEC material makes this point with a slide titled Artificial Intelligence Is Not a Superhero! [COMCEC], which is a better teaching device than a paragraph of caveats. Its function is to break the frame in which AI is imagined as a general capability that can be pointed at any problem. The correct frame is a set of instruments, each with a domain, a resolution, and a failure mode — closer to a workshop than to a mind.

4.3 The five-question test

Among the most useful passages in the source material is COMCEC’s formula for deciding whether to use AI at all. It poses five questions in order: (1) use cases — are there effective, measurable, quickly solvable problems? (2) skills — do you have the people to apply the techniques? (3) data — do you have relevant data for the chosen use cases? (4) technology — given your skills and data, which techniques can be applied? (5) organisation — how does this fit the organisation’s capacity to absorb it? [COMCEC].

The ordering is the lesson. Most failed projects begin at question four — someone chooses a technique and looks for a problem — and most successful ones begin at question one with a problem that is small enough to solve and important enough to matter. The query order is a diagnostic: if a proposal cannot answer questions one to three in plain language, the discussion of technique is premature.

I have used a variant of this test on every feature of the case in Part Four, and it is why Tapstall's AI surface is so narrow. The use case is small and measurable: find businesses a salesperson can call, with a reason to call. The data is public directory pages. The skills are one person's. The technology is HTTP requests and string matching, with a model in the loop only for drafting. The organisation is one person, so the system must not create work it cannot absorb — which is why the daily ceiling on new leads is twenty rather than two hundred.

4.4 Where the question is usually answered wrongly

Three failure patterns recur.

The demo problem. A capability demonstrated on curated examples is taken as evidence about production. The correction is a data audit before a model choice: what does the actual input distribution look like, including its ugly cases?

The definition problem. The task is defined in a way that makes the hard part invisible. "Classify this page as a business" hides the fact that most pages mentioning a business are not the business's page. The correction is to write the definition as an outcome a person will act on — "a sales-

person can open this and start a conversation" — and then to collect the ugly cases.

The absorption problem. The system produces more output than the organisation can consume, and the excess becomes guilt, then noise, then distrust. The correction is to cap throughput to human capacity, which is a design decision, not a technical one, and which Chapter Sixteen treats as a first-class constraint.

4.5 Summary

Capability lists date; limit lists do not. The engineering translation of the limits is distributional: competence is bounded by what the system has seen and by how the task was defined. The five-question test — use case, skills, data, technology, organisation — should be answered in that order, and a proposal that cannot answer the first three has not yet earned a discussion of technique.

Worked example — applying the test. A municipal office proposes an AI system to detect illegal dumping from photographs of streets. Apply the five questions. Use case: measurable, if the output is a list of locations for inspectors rather than prosecutions. Skills: who maintains the model when the seasons change the light? Data: do you have labelled photographs from this city, or only from the vendor's? Technology: is this a vision task or a workflow task in disguise? Organisation: can inspectors absorb the output, and who is accountable when the model is wrong? The third and fifth answers usually decide the project.

Exercises. (recall) List COMCEC's five decision questions in order and say why the order matters. (applied) Take a process you know and write both a poor definition and a

good definition of the AI task, where the difference is what failure becomes visible. (critical) “If an organisation cannot absorb a system’s output, the system is not ready.” Defend or attack, with an example.

Further reading. [COMCEC], “What Can Artificial Intelligence Do?” through “The Right Formula for Deciding on AI Use”; [Microsoft], “The Need for AI”; [NITI Aayog], Unit 1, “Why Do We Need AI?”.

CHAPTER 5

Intelligence in the small firm

Learning outcomes. By the end of this chapter you should be able to: explain why AI adoption in small firms differs structurally from adoption in large ones; describe the three constraints that bind hardest; and identify where AI is and is not worth deploying in a small business.

5.1 The unit of analysis is the operator, not the model

Everything in the source curricula is written for a learner, not for an operator. That is appropriate — they are educational materials — but it leaves a gap that this book exists to fill. A student learns what a neural network is; an operator needs to know whether to trust a supplier's claim, how to keep a small system running when the person who built it is on leave, and what to do when the data changes.

The difference is structural, and it has three parts.

Attention is the scarcest input. A large firm can hire a team to absorb a system's output. A small firm has one or two people who are also selling, delivering and keeping the books. A system that produces more than they can consume does not merely fail; it consumes the resource it was meant to save.

Capital is not the constraint; confidence is. Free and low-cost tools have removed the capital barrier for the software itself. What remains is the risk of spending scarce attention on something that does not work, and the reputational cost of a customer-facing failure.

The failure modes are social before they are technical. A model that is 95% accurate is a technical result. A model that is wrong in front of a customer — the wrong number, the wrong name, the wrong price — is a business event. The small firm experiences the second, not the first.

5.2 What small firms actually need from AI

Three things, in descending order of value.

Compression of dull work with a checkable output. Drafting a product description, summarising a month of orders, extracting a phone number from a page — tasks where the output can be inspected in seconds by the person who will use it.

A second opinion on a document. Composing a statement, a quote, a reminder, and having a system check it for omissions against a checklist. The value is in the checklist, not the prose.

Search across their own records. Once a merchant has three hundred orders, the ability to ask “what did I sell to this customer, and did they pay?” is worth more than any generative flourish.

Notice what is absent from the list: prediction of demand, automated pricing, customer sentiment dashboards. These are valuable in firms with data teams and useless in firms without them, because they produce outputs nobody has time to interpret.

5.3 The three constraints that bind

Data. A small firm’s data is thin and idiosyncratic. Two years of orders, several spellings of each customer’s name, prices in two currencies, and a catalogue that changes

seasonally. Microsoft's module discusses public datasets as a resource [Microsoft]; in a small firm, the relevant dataset is usually proprietary, small, and dirty, and the work is preparation rather than modelling.

Skills. The five-question test's second question is not a formality [COMCEC]. If the person maintaining the system cannot debug a failing API call, the system has a single point of failure that is also the business's owner.

Absorption. The organisation can consume a bounded amount of new information per day. Chapter Sixteen turns this into a hard ceiling.

5.4 The case in one paragraph

Tapstall is a merchant commerce rail run by one person on free infrastructure in four African markets. It gives a shop a storefront, an order link, a one-tap buyer confirmation and — the artefact that carries the business — a settlement statement reconciling orders against payments the merchant received directly. It holds no money, charges a flat fee, and uses AI narrowly: to draft copy, to read public pages while looking for prospects, and to summarise. Its most interesting design decisions are all restrictions — a gate that refuses candidates, a budget ladder that protects orders, a doctrine that forbids certain persuasive moves — and each restriction exists because the small-firm constraints above made the alternative unaffordable.

That is the shape of AI in the small firm, and it is the opposite of the shape in the large one. The large firm asks what a system can do and then builds controls. The small firm asks what it can safely ignore, and builds only what survives that question.

5.5 Summary

In a small firm, attention is the scarce input, confidence is the binding constraint, and failures are social before they are technical. The valuable applications compress dull work with checkable output, give a second opinion against a checklist, and search the firm's own records. Data is thin, skills are one person deep, and absorption is capped.

Worked example — the merchant's three questions. A shopkeeper considering a software subscription asks: will this save me time this week, will it work on the phone I have, and what happens if it stops working? Translate each into a design requirement. Answer: (1) the first useful output must arrive on day one, not after a setup project; (2) the interface must be a link that works in a browser on a low-end device; (3) the system must degrade to a state where the merchant still has her records — which is the architectural reason Tapstall's statement can be exported and recomputed.

Exercises. (recall) Name the three constraints that bind hardest on AI adoption in a small firm. (applied) For a small business you know, list three tasks that meet the "dull, frequent, checkable output" test and three that do not. (critical) "Most AI products sold to small businesses are solutions to problems those businesses do not have." Assess.

Further reading. [Microsoft], "The Need for AI"; [COMCEC], "The Right Formula for Deciding on AI Use"; Sarasvathy (2008) on reasoning from means at hand; Baker and Nelson (2005) on bricolage.

PART TWO

The Material

CHAPTER 6

Data: the raw material

Learning outcomes. By the end of this chapter you should be able to: classify data by type and structure; distinguish the five dimensions of data quality; and audit a small dataset for the defects that most often break a model.

6.1 Why data comes before algorithms

Every curriculum in this book's source set puts data before modelling, and the order is not arbitrary. COMCEC's treatment is explicit that the quality of a system's output is bounded by the quality of its input data, and that a large share of the practical work in AI is preparing data rather than choosing techniques [COMCEC]. Microsoft's module lists public sources and databases as a foundational resource, and discusses the storage layer — local versus cloud, and the categories of cloud service — before it discusses what models do with the data [Microsoft]. NITI Aayog devotes an entire unit of seven to data, and another to its visualisation [NITI Aayog].

The reason is arithmetic. A model is a function fitted to examples. If the examples are unrepresentative, the fitted function is confidently wrong on everything the examples did not cover — and confidence is the dangerous part, because a wrong answer delivered hesitantly is caught by a human, while a wrong answer delivered fluently is not.

6.2 Types of data

The standard taxonomy distinguishes:

Structured data — values in a fixed schema: rows and columns, fields with types. A merchant's order log is structured. Most business data is structured or aspires to be.

Semi-structured data — data with organisation but without a rigid schema: JSON, XML, tagged HTML. A directory page is semi-structured; so is an API response.

Unstructured data — text, images, audio, video. Most of the world's data is unstructured, and most of the recent progress in AI is progress on it, which is why the field's centre of gravity moved from databases to perception.

A second axis is measurement scale, and it matters because it constrains what operations are legal:

Scale	Property	Legal operations	Example
Nominal	Labels only, no order	Count, mode	Business category
Ordinal	Ordered, unequal gaps	Median, rank	Customer rating 1–5
Interval	Ordered, equal gaps, arbitrary zero	Mean, difference	Temperature in °C
Ratio	Ordered, equal gaps, true zero	All arithmetic	Revenue, order count

The common error is to treat an ordinal scale as interval. Averaging star ratings is arithmetically indefensible — the distance between one and two stars is not the distance between four and five — yet it is universal practice, because the resulting number is convenient and the error is small in the middle of the distribution. A student should be able to say why it is wrong even when choosing to do it.

6.3 Five dimensions of quality

Data quality is usually taught as a list of dimensions, and the list is worth memorising because it converts a vague worry into an audit.

Accuracy. Does the value match the world? A phone number that is well-formed and belongs to a different business is inaccurate. This is the dimension most often missed, because format checks pass.

Completeness. Are the required values present? Missing data is rarely random. In a merchant dataset, a missing delivery address correlates with orders that were never fulfilled, so dropping incomplete rows silently deletes the failures and inflates every downstream success rate.

Consistency. Do two records of the same thing agree? The same customer may appear as Tendai M. and Tendai Moyo in two systems, and a join that assumes consistency will double-count.

Timeliness. Is the value current? A directory listing for a business that closed eight months ago is timely in the sense that it was accurate when captured and useless for the purpose at hand.

Validity. Does the value conform to the rules of its domain? A Zimbabwean mobile number has a particular shape; a value that does not conform is invalid whether or not it is accurate.

An audit instrument follows directly: for each field, ask all five questions and record the failure rate on a sample of a hundred rows. The exercise takes under an hour and routinely kills projects that would otherwise have con-

sumed months — which is precisely what Chapter Four's second failure pattern predicted.

6.4 The defect that ruins models: leakage

Data quality has a dimension that the standard lists omit, and it is the one that most often produces a model that is magnificent in testing and worthless in production.

Leakage is the presence, in the features used for prediction, of information that would not be available at prediction time — or that is a consequence of the outcome rather than a cause of it. If you build a model to predict whether an order will be paid, and one of your features is "date the payment was recorded", your model will achieve perfect accuracy on historical data and be useless the moment it is deployed, because the payment date does not exist until after the outcome.

Leakage is insidious because it is invisible in the metric. The accuracy is real; the setting in which it was measured is not. Students find this hard because it feels like cheating, and it is: you have allowed the future into the training set. The discipline that prevents it is to write, for each feature, the moment at which it would be known, and to require that moment to precede the prediction.

A second, subtler form is selection leakage. If you assemble a dataset of businesses that are customers of your product, the features that describe them encode the fact that they were willing to buy. A model trained on that set will not find prospects; it will find the correlates of past persuasion.

6.5 Representativeness

The last property to check is whether the dataset resembles the world the system will meet. Three failure modes recur.

Coverage. If your corpus of a language contains only formal news text, a system trained on it will fail on the informal, code-switched language of a customer message. The failure is not evenly distributed: it falls hardest on exactly the users least served by existing systems, because their data is least represented.

Base rate drift. The proportion of positives in your training set may not match the proportion in production. A model trained where 50% of pages are businesses will behave differently on a feed where 4% are.

Cohort drift. The world changes. A model trained on last year's listings will slowly describe last year's market. The mitigation is not a better model but a monitoring plan, which Chapter Thirteen develops.

6.6 Summary

Data comes first because a model is a function fitted to examples. Classify data by structure and by measurement scale, because the scale determines legal operations. Audit five dimensions — accuracy, completeness, consistency, timeliness, validity — and add two the standard list omits: leakage and representativeness. Leakage produces models that are perfect in testing and worthless in production, and representativeness determines who the system fails.

Worked example — a hundred-row audit. You have 100 export rows from a merchant's order log: `order_id, date, customer, phone, items, total, paid`. Design the audit. An-

swer: accuracy — spot-check ten phone numbers by calling; completeness — count missing phone, missing total; consistency — count duplicate order IDs and customers appearing with variant spellings; timeliness — check the date range and whether the log is current; validity — check currency codes, negative totals, phone formats per country. Then the two extras: leakage — does `paid` appear anywhere you might use as a feature? representativeness — what fraction of orders in the period are in the export at all, and were the exclusions systematic?

Exercises. (recall) List the five standard dimensions of data quality and give one failure of each from a small business context. (applied) Take a spreadsheet you have access to with at least fifty rows and produce a quality scorecard: dimension, field, failure rate, consequence. (critical) “Dirty data is not an obstacle to AI in developing markets; it is the whole problem.” Discuss with reference to coverage and representativeness.

Further reading. [NITI Aayog], Unit 3, “Data”; [COMCEC], “Data and AI”; [Microsoft], “Public Databases and Datasets”; Redman (2008) on data quality.

CHAPTER 7

Preparing data: the lifecycle

Learning outcomes. By the end of this chapter you should be able to: name the stages of the AI lifecycle as the source curricula present them; explain what happens at each stage and what can go wrong there; and produce a lifecycle plan for a small deployment.

7.1 Six stages

COMCEC presents the work of building an AI system as a sequence of stages, beginning with data collection and proceeding through data operations, modelling and development, deployment, results collection and validation [COMCEC]. The sequence is worth learning precisely, because it names the activities that proposals omit when they promise a model in six weeks.

1. Data collection. Acquire raw material: scrape, export, purchase, or solicit. Requirements: lawful basis, representativeness, and a record of provenance — where each value came from, so that an audit can go back.

2. Data operations. Clean, transform, integrate, and label. This is the largest stage in most real projects. It includes de-duplication, normalisation of formats, resolution of entities, handling of missing values, and the construction of labels where none exist.

3. Modelling and development. Choose a technique, fit it, tune it, and compare it against a baseline. The baseline is the stage’s most neglected element: a system that beats a trivial rule by two percentage points is not worth deploy-

ing, and you only know this if you computed the trivial rule first.

4. Deployment. Put the system where the work happens. Requirements: latency the user will tolerate, failure behaviour when a dependency is down, cost per call, and a way to turn it off.

5. Results collection. Measure what the deployed system produces. This stage is where most projects quietly stop, because measurement requires instrumentation and instrumentation is nobody's favourite task.

6. Validation. Compare results against the goal. Did the system solve the problem it was built for — not the problem it was measured on?

The sequence is a loop, not a line. Validation reveals that the labels were wrong, which returns you to stage two; results collection reveals that the deployment environment differs from the development one, which returns you to stage one. A student who treats the diagram as a waterfall will be surprised by their first real project.

7.2 The distribution of effort

Across the systems described in Part Four, the effort split was roughly: collection and operations 60%, modelling 10%, deployment 20%, measurement and validation 10%. The exact numbers are not the point; the shape is. A project plan that allocates 70% of its time to modelling has allocated its time backwards.

This is not a peculiarity of small systems. It is the general condition of applied machine learning, and it is why the

phrase “just plug in a model” describes the smallest part of the work.

7.3 Cleaning operations, concretely

Six operations cover most of what a small dataset needs, and each has a characteristic trap.

De-duplication. Two rows describing the same entity. Trap: naive matching on a single field merges distinct entities (two businesses genuinely called Blessed Hardware) and misses true duplicates (the same business with a typo). Use blocking — compare only records that share a blocking key — then compare on several fields with fuzzy matching, and keep a human in the loop for merges above a similarity threshold.

Normalisation. The same value in different forms: `+263 77 123 4567`, `0771234567`, `00263771234567`. Trap: normalising to a target format that loses information, such as dropping the country code on numbers that may be international.

Missing values. Options: drop rows, drop columns, impute a constant, impute a statistic, or model the missingness. Trap: dropping rows when missingness is informative (see §6.3). The honest approach is often to add an explicit was_missing indicator alongside the imputed value, so the model can learn whether absence itself predicts the outcome.

Outliers. Values far from the rest. Trap: removing them automatically. An outlier may be a data error or may be the most important case in the dataset — the order for ten thousand units, the customer who pays late by ninety days. Investigate before deleting.

Type coercion. Text that should be numeric. Trap: silent coercion to zero for unparseable values, which converts "we do not know" into "it is nothing" — a substitution with real consequences in any financial field.

Encoding. Converting categories for a model. Trap: applying ordinal encoding to nominal categories, inventing an order that does not exist. Chapter Eight develops the alternatives.

7.4 Labelling, the expensive part

Supervised learning needs labels, and labels come from people. Four rules make the process survivable.

Write the definition before you label. A labelling guide of one page, with examples, that two people can apply independently and agree on. If they cannot agree, the task is under-specified and no model will rescue it.

Measure agreement. Label a hundred items twice and compute the agreement rate. If two humans agree on 80% of items, a model cannot reasonably be expected to exceed about 80% — the ceiling is in the definition, not the algorithm.

Label blind to the hypothesis. A labeller who knows what the model predicted will drift toward agreement. Hide it.

Keep a held-out set untouched until the end. Every inspection of the test set during development leaks information into your decisions, and after twenty inspections the set has stopped being held out.

7.5 The lifecycle in a one-person firm

In a firm of one, the lifecycle must be compressed without being skipped.

Collection is an HTTP request and a saved JSON file, with the URL and date written into the record so that provenance is free.

Operations is a script in version control, so that a transform can be re-run and a bug can be fixed once.

Modelling is often a rule plus a threshold, and the baseline is a spreadsheet column that says yes or no.

Deployment is a scheduled job with a fixed daily ceiling, so that a bug cannot flood the queue.

Results collection is a table of outcomes the person must record anyway for their own purposes — a discipline that makes measurement a by-product of ordinary work rather than a separate project.

Validation is a monthly reading of that table against the question the system was built to answer.

Each row of that paragraph corresponds to a design decision in Part Four, and none of them is about model choice.

7.6 Summary

The lifecycle is collection, operations, modelling, deployment, results collection, validation — a loop, not a line. Effort concentrates in collection and operations. Cleaning has six common operations, each with a trap. Labelling is the expensive part and its quality ceiling is set by inter-

rater agreement on a written definition. In a small firm, compress the stages, do not skip them.

Worked example — costing a pipeline. You want to classify 2,000 directory pages. Estimate the labour. Answer: collection is automated (hours). Operations: extract text, strip navigation, remove boilerplate — one day. Labelling: at 30 seconds per page, 2,000 pages is about 17 hours, plus the guide, plus a double-labelled sample of 100 for agreement — call it three days. Modelling: two days including a baseline. Deployment: a day. Measurement: ongoing. The estimate is nine working days, of which seven are data work. Any plan that says "a week" has assumed the data work away.

Exercises. (recall) List the six lifecycle stages in order and name the characteristic failure of each. (applied) Write a one-page labelling guide for a binary task in your domain, then have a classmate apply it to twenty items; compute agreement. (critical) "Collecting more data is almost always better than choosing a better model." Argue both sides with reference to diminishing returns and representativeness.

Further reading. [COMCEC], the lifecycle and preprocessing sections; [NITI Aayog], Unit 3; Sculley et al. (2015) on hidden technical debt in machine learning systems.

CHAPTER 8

Features: the representation that decides the outcome

Learning outcomes. By the end of this chapter you should be able to: explain what a feature is and why representation determines achievable accuracy; apply at least five feature-construction techniques; and recognise the leakage risks introduced by feature engineering.

8.1 The claim

Andrew Ng's remark, quoted in the COMCEC material, is the chapter's thesis: "Applied machine learning is basically feature engineering" [COMCEC]. The claim is that the choice of representation — which quantities you compute from raw data and hand to a learner — accounts for more of a system's performance than the choice of learner.

The reason is that a learner can only find structure that is visible in its inputs. If the difference between a business page and a directory listing lies in the presence of a street address near a phone number, and your feature set is a bag of individual words, that relational fact is invisible, and no amount of model capacity will recover it — the information was discarded before the model saw it.

8.2 The basic transformations

Normalisation and scaling. Put numeric features on comparable ranges. Min-max scaling maps to [0,1]; standardisation maps to mean zero and unit variance. Needed for methods that compute distances (k-nearest neighbours, clustering, support vector machines with certain kernels)

and for gradient-based training stability. Not needed for decision trees, which split on thresholds and are invariant to monotone transforms.

Encoding categorical variables. Three options. One-hot: one binary column per category, which invents no order and explodes in width when a category has thousands of values. Ordinal: integers, which invents an order and should be reserved for genuinely ordered categories. Target or frequency encoding: replace the category with a statistic computed from the training data — dangerous, because it leaks the target unless computed per fold.

Binning. Convert a continuous variable into intervals. Useful when the relationship is non-linear and step-shaped, and when interpretability matters.

Log transforms. Compress heavy-tailed quantities such as order values, where a few large values would otherwise dominate. The transform is monotone, so ranking is preserved.

8.3 Relational features, and why they matter most

The features that carry signal in messy real data are usually relational: they describe how parts of a record relate to each other, or how a record relates to its cohort.

For a business page, compare two representations:

Representation	Features	What the model can see
Bag of words	counts of the 500 most common words	topic, weakly
Structural	has street address; phone within N characters of an address; count of distinct phone numbers; length of the page's link text; presence of opening hours; ratio of links to words	whether the page is a business, or merely mentions one

The second representation is smaller and vastly more useful, and note that it required no learning at all — it required looking at what distinguishes the classes. That looking is the work.

Interaction features multiply or combine two variables to expose a joint effect: `order_value × days_since_last_order` may predict churn far better than either alone. **Ratio features** normalise by scale: orders per month rather than total orders, so that older merchants are not automatically ranked higher. **Aggregate features** describe a record's group: this customer's average order value relative to the merchant's, a business's listing count relative to its category.

8.4 Time, the hardest feature

Temporal features require care because they are the most common source of leakage.

Point-in-time correctness. Every feature must be computed using only data available before the prediction moment. A customer's "lifetime value" computed across the

whole dataset includes the future, and a model trained on it will look superb and fail in production. The discipline is to build features from a window that ends at the prediction time, for every training example separately — which is more expensive than a single group-by and is therefore skipped, which is therefore why leakage is ubiquitous.

Recency, frequency, monetary value (RFM). A standard and durable feature set for commerce: how recently did this customer buy, how often, and how much. Each is a windowed aggregate, so each must be computed point-in-time.

Seasonality and calendar effects. Day of week, month, and proximity to holidays. In a market where salaries are paid at month end, the day of the month is a powerful feature and is invisible to a model given only timestamps.

8.5 Feature selection and the discipline of fewer

More features are not better. Each feature is a liability: it can drift, it can leak, it can be expensive to compute, it needs monitoring, and it adds a dimension in which the model can find spurious structure.

Three selection strategies. **Filter** methods rank features by a univariate statistic against the target. **Wrapper** methods evaluate subsets by model performance — accurate and expensive. **Embedded** methods let the model choose, as with the coefficient penalties of L1 regularisation.

The practical rule for small datasets: keep the features you can explain to the person who will act on the output. A feature you cannot explain cannot be defended when the model is wrong, and in a small firm the defence happens in front of the customer.

8.6 Summary

Representation bounds performance, because a learner cannot see structure that its inputs do not contain. Scaling, encoding, binning and log transforms are the elementary operations; relational and interaction features are where the signal in messy data usually lives; temporal features are the hardest and the most common source of leakage. Fewer, explainable features beat many unexplainable ones in small deployments.

Worked example — engineering a churn feature. You have orders with `customer_id, date, value`. Build features to predict whether a customer orders again next month, without leaking. Answer: for each customer and each prediction date t: recency = days since last order before t; frequency = count of orders in the 90 days before t; monetary = mean order value in that window; trend = mean of the last three orders' values minus the mean of the three before; regularity = standard deviation of inter-order intervals; and missing indicators for customers with no prior orders. Every one uses only data before t. Then a baseline: "predict yes if recency < 30 days". If your model cannot beat that baseline, the features are not earning their complexity.

Exercises. (recall) Give three reasons a feature is a liability. (applied) Take a small table and construct five features, annotating for each the moment at which it becomes known. (critical) "If you cannot explain a feature to the decision-maker, delete it." Discuss with reference to cases where opaque features genuinely help.

Further reading. [COMCEC], “Feature Engineering”; Zheng and Casari (2018); Ng’s remarks on applied machine learning as cited in [COMCEC].

CHAPTER 9

Seeing data: descriptive statistics and visualisation

Learning outcomes. By the end of this chapter you should be able to: summarise a distribution with the right statistics; choose a chart appropriate to the question and the data type; and identify three ways charts mislead.

9.1 Why this is in an AI book

NITI Aayog gives data visualisation its own unit among seven [NITI Aayog], and the inclusion is well judged. Before any model exists there is an exploratory step, and its purpose is not decoration: it is to discover that the data is not what you assumed. A histogram reveals that revenue is bimodal; a scatter plot reveals that two variables are related only above a threshold; a time series reveals a level shift that a summary statistic would average away.

The exploratory step is also where a project is most cheaply killed. Every hour spent looking at distributions is an hour not spent building a model for a relationship that does not exist.

9.2 Summary statistics, and their failure modes

Centre. Mean, median, mode. The mean is sensitive to skew and outliers; the median is not; the mode is meaningful only for discrete or binned data. For income, order value, or session duration — all heavy-tailed — the median is the honest summary and the mean is the marketing summary.

Spread. Range, interquartile range, variance, standard deviation. The standard deviation assumes a roughly symmetric distribution; for skewed data, report the IQR or quantiles.

Shape. Skew and kurtosis, or simply the histogram. Two datasets with identical means and standard deviations can look nothing alike — Anscombe's quartet and its successor, the datasaurus dozen, exist to make this point unforgettable.

Relationships. Covariance and correlation. Correlation coefficients measure linear association, so a strong nonlinear relationship can show a coefficient near zero. And correlation is not causation — the standard warning, but its operational form is more useful: a correlation between two variables may be produced by a third variable that causes both, by selection into the sample, or by the passage of time.

9.3 Choosing a chart

The choice follows from the question, not from taste.

Question	Chart	Note
What is the distribution of one variable?	Histogram, box plot, violin	Box plots hide multimodality; use a histogram when shape matters
How do two variables relate?	Scatter plot	Add a trend line only if the relationship is plausibly monotone
How does a value change over time?	Line chart	Never use a line for categorical data; the line implies continuity
How do categories compare?	Bar chart, sorted	If parts of a whole, use a stacked bar or a treemap rather than a pie
What is the relationship among many variables?	Correlation heatmap, pair plot	With more than about eight variables, use a matrix, not a grid of all pairs

Two rules that survive every style guide. **The axis starts at zero for bar charts**, because bar length encodes magnitude. **The axis need not start at zero for line charts**, because position encodes value and the interesting variation may be small — but say so, since readers assume truncation is manipulation.

9.4 Three ways charts mislead

Dual axes. Two vertical scales on one plot allow any two series to be made to look correlated. If the correlation matters, plot the relationship directly; if it does not, do not plot two axes.

Aggregation. A trend in totals can conceal an opposite trend in every component — Simpson’s paradox. Check the

components before reporting the aggregate, particularly when the aggregate is the number someone wants to see.

Colour without meaning. Encoding a third variable in colour when the colour carries no information, or using a rainbow scale for an ordered quantity, which misleads the eye about magnitude. Use a perceptually uniform scale, and use one.

9.5 Visualisation as an audit

The deepest use of charts in applied work is not communication but inspection. Four plots, run before any model, catch a large share of problems.

Target distribution. Is the class balance what you expected? A ten-to-one imbalance changes everything downstream.

Missingness map. Which fields are missing, and do the missing patterns cluster in time or in cohort? A field that is missing only after a certain date is a system change, not a data defect.

Feature-versus-target plots. For each feature, plot its distribution split by class. If a single feature separates the classes almost perfectly, suspect leakage before celebrating.

Time series of volume. Records per day. Gaps are collection failures; spikes are duplicates or a scrape that ran twice.

Each of these takes minutes and pays for itself repeatedly. In the case, the missingness map was the plot that mattered: a batch of prospect records had a phone field populated from a page’s navigation rather than its body,

and the pattern was visible as a cluster rather than scattered noise.

9.6 Summary

Exploration is cheap and kills bad projects early. Choose centre and spread statistics that survive skew, and correlate with an awareness of non-linearity. Choose charts from the question, honour the zero-baseline rule for bars, and avoid dual axes, unexamined aggregation and decorative colour. Four audit plots — target balance, missingness, feature-versus-target, record volume over time — should precede every model.

Worked example — the bimodal revenue plot. A merchant's monthly revenue histogram shows two peaks, at roughly $40 and $260. What does this mean and what do you do? Answer: the population is a mixture of two kinds of month or two kinds of merchant — for instance cash-flow months and restocking months. A single mean describes neither. Split the data by the plausible grouping variable, re-plot both, and check whether the split is real before modelling. If it is real, the model needs the grouping variable as a feature, or needs to be two models.

Exercises. (recall) For skewed data, which measure of centre and which measure of spread should you report, and why? (applied) Produce the four audit plots for any dataset and write a half-page finding for each. (critical) Find a published chart that misleads and write a 200-word critique identifying the specific mechanism.

Further reading. [NITI Aayog], Unit 4, "Data Visualisation"; Tufte (2001); Cairo (2016); Anscombe (1973).

CHAPTER 10

Learning from data

Learning outcomes. By the end of this chapter you should be able to: distinguish supervised, unsupervised and reinforcement learning; explain the bias–variance trade-off; describe how a model is fitted and what overfitting looks like; and choose an appropriate learning paradigm for a stated problem.

10.1 The three paradigms

Every curriculum in the source set presents the same tripartite division, and NITI Aayog's Unit 2 develops it at length [NITI Aayog]; Microsoft's module reaches it through applications [Microsoft]; COMCEC places it in the modelling stage of the lifecycle [COMCEC].

Supervised learning. The data consists of input–output pairs, and the task is to learn the mapping. Two sub-tasks. Classification predicts a discrete label: is this page a business, yes or no; which of five categories does this complaint belong to. Regression predicts a continuous value: what will this order be worth, how many days until this customer returns.

Unsupervised learning. The data consists of inputs only, and the task is to find structure. Clustering groups similar records; dimensionality reduction compresses many features into few; anomaly detection identifies records unlike the rest.

Reinforcement learning. An agent takes actions in an environment and receives rewards; the task is to learn a

policy that maximises cumulative reward. The distinctive feature is that the data is generated by the agent's own behaviour, so learning and acting are entangled — the agent must explore to learn and exploit to earn, and the balance between them is the field's central problem.

A fourth category sometimes appears: semi-supervised learning, where a small labelled set is combined with a large unlabelled one. It is the honest description of most real deployments, where labels are expensive and raw data is abundant.

10.2 Fitting: what actually happens

A supervised model is a function with parameters. Fitting is the search for parameter values that minimise a loss — a number measuring how wrong the model's predictions are on the training data. For classification, a common loss is cross-entropy; for regression, squared error.

Training proceeds by gradient descent: compute the gradient of the loss with respect to each parameter, and take a small step in the direction that reduces it. Repeat. The learning rate controls step size: too small and training takes forever; too large and the process oscillates or diverges.

Three datasets, and the discipline of keeping them apart:

- **Training set** — used to fit parameters.
- **Validation set** — used to choose hyperparameters and compare models.
- **Test set** — used once, at the end, to estimate performance on unseen data.

Every decision informed by a dataset contaminates it. If you tune against the test set, your reported performance is optimistic, and the optimism is largest exactly where you most need a reliable estimate — on the hard cases you kept adjusting for.

10.3 Overfitting and underfitting

Underfitting is a model too simple for the structure in the data: it performs poorly on training and test data. **Overfitting** is a model too flexible: it performs well on training data and poorly on test data, because it has learned the noise.

The formal framing is the **bias–variance trade-off**. A model's expected error decomposes into bias (systematic error from wrong assumptions), variance (sensitivity to the particular training sample), and irreducible noise. Simple models have high bias and low variance; flexible models have low bias and high variance. Total error is U-shaped in model complexity, and the practical task is to find the bottom of the U.

Four countermeasures, in rough order of value for small datasets:

More data. The most reliable reducer of variance. Not always available.

Regularisation. Add a penalty on parameter magnitude to the loss, discouraging the model from fitting noise. L2 (ridge) shrinks parameters smoothly; L1 (lasso) drives some to zero and performs selection.

Early stopping. Monitor validation loss during training and stop when it begins to rise.

Cross-validation. With limited data, split into k folds, train on k−1 and validate on the held-out fold, rotating, then average. Five or ten folds is standard. Report the mean and the spread: a model with 80% mean accuracy and a range of 65–92% across folds is not a model you can rely on.

10.4 The algorithms a first course should know

Linear and logistic regression. Simple, interpretable, and a strong baseline. Logistic regression maps a linear combination of features through a sigmoid to a probability in (0,1). Understanding it is understanding what "coefficient" and "odds" mean, which is most of what is needed to read the applied literature critically.

Decision trees. Recursive partitioning on feature thresholds. Interpretable, handles non-linearity and interactions, does not require scaling. Prone to overfitting when deep; ensembling fixes this.

Ensembles. Random forests average many decorrelated trees, reducing variance. Gradient boosting fits trees sequentially to the residual errors of the previous ensemble, reducing bias. Both are the workhorses of tabular prediction and both are strong defaults.

k-nearest neighbours. Classify by the majority label of the k closest training points. Requires scaling; degrades in high dimensions; excellent as an intuitive teaching device for the idea that "similar inputs have similar outputs".

Support vector machines. Find the separating hyperplane with maximum margin; kernels allow non-linear boundaries. Effective on small, clean datasets.

Neural networks. Layers of weighted sums and non-linear activations, fitted by backpropagation. The universal approximation property says a sufficiently wide network can approximate any continuous function — a statement about existence, not about learnability from finite data, and the distinction is the most commonly abused fact in the field's popular literature.

Clustering. k-means iteratively assigns points to the nearest of k centroids and recomputes; it requires k in advance and assumes roughly spherical clusters. Hierarchical clustering builds a tree of merges, which lets you choose the number after the fact.

10.5 Choosing a paradigm

The choice follows from what you have, not from what is fashionable.

You have	You want	Use
Labelled examples	Predict the label	Supervised classification
Labelled numeric outcomes	Predict the value	Supervised regression
Unlabelled records	Find groups	Clustering
Unlabelled records	Find the odd ones	Anomaly detection
A sequence of decisions with delayed feedback	Learn a policy	Reinforcement learning
A few labels, much raw data	Extend the labels	Semi-supervised

Reinforcement learning is the paradigm most often proposed for problems it does not fit. It requires an environment in which the agent can act repeatedly, a reward signal that is not too delayed, and tolerance for a learning period during which performance is poor. A small business has none of these: its actions are irreversible (you cannot un-call a customer), its reward is delayed by weeks, and it cannot afford an exploration phase.

10.6 Summary

Supervised learning maps inputs to outputs; unsupervised finds structure; reinforcement learns a policy from interaction. Fitting minimises a loss by gradient descent, and honesty requires three separate datasets. Overfitting is the dominant failure and its countermeasures are more data, regularisation, early stopping and cross-validation. Choose the paradigm from the data you have, not the prestige of the technique.

Worked example — diagnosing a model. Training accuracy 99%, test accuracy 71%. What is happening and what do you try, in what order? Answer: the gap is variance — the model has memorised the training set. Order of attempts: (1) audit for leakage — 99% is suspiciously high and leakage is more likely than genuine fit; (2) if clean, reduce model complexity or add regularisation; (3) get more data if possible; (4) use cross-validation to get a reliable estimate before and after each change. Do not start by tuning.

Exercises. (recall) Define bias and variance and give one countermeasure for each. (applied) Fit logistic regression and a random forest to the same tabular dataset; report cross-validated accuracy with spread, and explain the dif-

ference. (critical) “Reinforcement learning is the wrong paradigm for almost every business problem.” Discuss.

Further reading. [NITI Aayog], Unit 2, “Learning”; [COMCEC], “Machine Learning”; Hastie, Tibshirani and Friedman (2009), chapters 2 and 7; Sutton and Barto (2018), chapter 1.

CHAPTER 11

Measuring a model honestly

Learning outcomes. By the end of this chapter you should be able to: construct and read a confusion matrix; compute precision, recall and F1 by hand; explain why accuracy is misleading under class imbalance; and specify an evaluation plan for a deployment.

11.1 The confusion matrix

Almost every classification metric in use is a summary of a two-by-two table. For a binary task with a positive class of interest:

	Predicted negative	Predicted positive
Actually negative	True negative (TN)	False positive (FP)
Actually positive	False negative (FN)	True positive (TP)

The four cells are not interchangeable, because the two kinds of error have different consequences. A false positive in a spam filter costs a lost message; a false negative costs an annoyance. A false negative in a cancer screen costs a life; a false positive costs an anxious week of tests. The matrix is where a task's values enter its measurement, and any single-number metric hides that entry.

11.2 The metrics and what each one hides

Accuracy = (TP + TN) / total. Honest only when the classes are balanced and the costs symmetric. With a 1% positive rate, a model that predicts "negative" always achieves 99% accuracy and is worthless. This is the single

most important fact in applied evaluation, and it is why the base rate must be reported alongside any accuracy figure.

Precision = TP / (TP + FP). Of the items the system flagged, what fraction were right? This is the metric that governs whether users trust the system, because users experience flagged items.

Recall (sensitivity) = TP / (TP + FN). Of the items that should have been flagged, what fraction were? This is the metric that governs missed opportunity or missed harm.

Specificity = TN / (TN + FP). Of the genuinely negative items, what fraction were correctly left alone.

F1 = the harmonic mean of precision and recall, 2·P·R/(P+R). Convenient for comparing systems when both errors matter equally; misleading when they do not, because it silently assumes they do.

Precision–recall trade-off. For most models there is a threshold knob: lowering it flags more items, raising recall and lowering precision. There is no configuration that maximises both, so a performance claim without a stated threshold is incomplete.

ROC and AUC. The receiver operating characteristic plots true-positive rate against false-positive rate across thresholds; the area under it is the probability that a randomly chosen positive is scored above a randomly chosen negative. Useful as a threshold-free summary, misleading under heavy imbalance, because a large number of negatives makes false-positive rates look small. For imbalanced problems, report the precision–recall curve and average precision instead.

11.3 A worked computation

Suppose a prospect-scoring model is evaluated on 1,000 pages of which 40 are genuine prospects.

	Predicted: not a prospect	Predicted: prospect
Actually a prospect	6	34
Actually not	940	20

Then: accuracy = (940 + 34)/1,000 = 97.4%. Precision = 34/(34+20) = 63.0%. Recall = 34/40 = 85.0%. Specificity = 940/960 = 97.9%. F1 = 2(0.630)(0.850)/(0.630+0.850) = 0.724.

Read the numbers as a person would experience them. Of every hundred pages the system puts in front of a salesperson, thirty-seven are wrong. That is the number that determines whether the salesperson keeps using the system — and it is invisible in the 97.4%.

Now consider the baseline: flag every page. Accuracy = 100% in the sense that all 40 prospects are caught, precision = 4%, and the queue is 1,000 items long. The naive model scores better on recall than the system and is useless. This is why **the baseline must always be reported**, and why the comparisons that matter are against trivial rules, not against other models.

11.4 Calibration, the neglected property

A model is calibrated if, among the items it scores at probability p, about a fraction p are positive. A model can rank well and be badly calibrated: it may place all the right

items at the top and still claim 90% confidence for items that are positive 60% of the time.

Calibration matters wherever the score drives a resource decision — how many leads to work, how much stock to hold, what to quote — because these decisions use the probability's level, not just its order. The diagnostic is a reliability plot: bin predictions, plot observed frequency against mean predicted probability. The remedy is usually a post-hoc mapping fitted on held-out data, such as Platt scaling or isotonic regression.

11.5 Evaluation plans

An evaluation plan is a short document written before modelling, and it has seven parts.

1. **The decision the model informs.** Not "classify pages" but "which twenty records a person works today".
2. **The unit of evaluation.** Records, or days, or customers? Evaluating per record when the decision is per day gives the wrong answer, because the errors cluster.
3. **The metric and the threshold.** Named explicitly, with the operating point fixed in advance, so that it cannot be chosen after seeing the results.
4. **The baseline.** The trivial rule, stated and measured.
5. **The split.** How train, validation and test were constructed, and how temporal ordering was respected.
6. **The slice analysis.** Which subgroups must be checked separately — by language, by country, by device, by merchant size — because an aggregate metric can conceal a subgroup where the system fails completely.

7. **The stopping rule.** What result would cause you not to deploy.

Part six is the one most often omitted and the one that most often matters. A system with 90% overall accuracy that achieves 55% on the language spoken by a third of its users is not a system with a language problem; it is a system that fails a third of its users, and the aggregate concealed it.

11.6 Summary

Metrics are summaries of a confusion matrix, and the matrix is where the task's costs enter. Accuracy misleads under imbalance; precision describes trust, recall describes misses; F1 assumes symmetric costs. Calibration matters when the score's level drives a decision. Write the evaluation plan before modelling, including the baseline, the slices and the stopping rule.

Worked example — choosing a metric. A system flags orders that look fraudulent for manual review. Which metric? Answer: the constraint is review capacity, so the decision variable is precision at a fixed review budget — of the top 50 flagged per day, how many are truly fraudulent. Recall is secondary but not irrelevant: report how much fraud the 50-item budget misses. Accuracy is irrelevant; the base rate is far too low.

Exercises. (recall) Write the formulas for precision, recall and F1. (applied) Reproduce the worked computation above for a threshold that flags 100 pages, and plot precision and recall at five thresholds. (critical) "A model nobody uses because its precision is 60% is worse than a

rule that everyone uses because they understand it.” Discuss.

Further reading. Saito and Rehmsmeier (2015) on precision–recall versus ROC; Niculescu-Mizil and Caruana (2005) on calibration; [COMCEC], on evaluating models.

CHAPTER 12

The arithmetic of being wrong

Learning outcomes. By the end of this chapter you should be able to: compute the expected cost of a classifier from its confusion matrix and the costs of its errors; choose a threshold from costs rather than from a default; and explain why a base rate can make an accurate system useless.

12.1 Turning a matrix into money

Classification metrics are dimensionless. Decisions are not. The bridge is expected cost.

For a binary classifier with error costs c_{FP} (the cost of a false positive) and c_{FN} (the cost of a false negative), the expected cost per item is

> $E[\text{cost}] = P(\text{FP}) \cdot c_{FP} + P(\text{FN}) \cdot c_{FN}$

and the threshold that minimises it is the one where the marginal cost of flagging equals the marginal cost of not flagging. In practice: sort items by predicted probability, and walk down the list, flagging while the expected benefit of flagging exceeds the cost. Everything else — every discussion of which algorithm to use — is subordinate to this calculation, because this is the calculation the business actually performs whether or not it does so explicitly.

12.2 Worked example: the cost of a bad lead

A salesperson can work twenty records a day. A prospect record, worked properly, takes twenty minutes including

the call and the notes. A good record yields a conversation worth, in expectation, some value v; a bad record yields nothing and costs the twenty minutes.

Let p be the precision of the queue. The expected value of working a record is p·v − c, where c is the twenty minutes valued at the salesperson's rate. Flagging a record is worth doing while p·v > c, so the minimum acceptable precision is

> p* = c / v

If the salesperson's time is worth $15 an hour and a conversation is worth $120 in expectation, then c = $5 and v = $120, so p* = 4.2%. That seems permissive, and it is — until you add the cost of erosion. A queue at 20% precision wastes four out of five attempts, and a salesperson who wastes four attempts in five stops using the queue. The cost of lost adoption is not in the formula and dominates everything in it.

Add the reputational term and the threshold changes qualitatively. Cold-calling a charity, a competitor, or a business that has asked not to be contacted is not merely unproductive; it is a harm with a cost that is not denominated in minutes. The correct treatment is to make such items inadmissible rather than expensive — a filter, not a penalty. Chapter Seventeen describes exactly this, and it is the design decision that turned a list of page-scrapings into a system.

12.3 The base-rate trap

The classic demonstration, in a medical screening frame, is worth reproducing because it recurs in every domain.

A test has 99% sensitivity and 99% specificity. The condition affects 1 in 10,000 people. A patient tests positive. What is the probability she has the condition?

Take 10,000 people. About one has the condition and tests positive with probability 0.99 — call it 1 true positive. The 9,999 without it generate about 100 false positives at a 1% rate. So a positive result corresponds to a 1-in-101 chance of disease: under 1%. The test is accurate and the result is nearly uninformative, because the base rate is small relative to the error rate.

The business analogue is exact. A fraud model at 99% specificity applied to a stream with a 0.1% fraud rate generates roughly ten false alarms per true one. A lead-quality model at 95% specificity on a stream where 2% of records are prospects generates more false alarms than true finds. **The base rate is a property of the deployment, not of the model**, and it is the first thing to compute when someone reports a specificity.

12.4 Thresholds are governance

Because the threshold determines how many items are flagged, it determines how much human work the system creates. This makes threshold selection a governance decision with three properties.

It is not technical. Two engineers with the same model and different staffing levels should choose different thresholds. A threshold set without reference to capacity will either starve the capacity or flood it.

It should be written down. The threshold, the reasoning, and the date. Otherwise it will be changed silently when

someone finds the queue annoying, and the change will be invisible in exactly the way Chapter Six described.

It should be reassessed when the base rate moves. A threshold tuned for a 40% positive rate is wrong for a 5% rate. Seasonal effects, market changes and new sources all move the base rate, and the threshold has to move with them.

12.5 Asymmetric errors in the field

Three examples from commerce, each with a different asymmetry.

Settlement reconciliation. An order marked as paid when it was not is a financial loss with a legal consequence; an order marked unpaid when it was paid is an annoyance resolved by a phone call. The asymmetry is severe, and the correct design is to require evidence for the paid state and to let the unpaid state be a default. Tapstall's statement works this way: the merchant records payments received directly, and the statement reconciles, rather than the platform assuming.

Fraud screening. Depends entirely on the intervention. If the intervention is a review queue, false positives are cheap and the threshold should be low. If the intervention is an account freeze, false positives are expensive and the threshold must be high. Same model, different threshold, because the action differs.

Content moderation for a small seller. A false positive removes a legitimate listing and costs a sale and goodwill; a false negative leaves a problematic listing up. The asymmetry depends on the platform's obligations, which is why

the threshold is a policy question rather than an engineering one.

12.6 Summary

Turn the confusion matrix into expected cost, and choose the threshold where marginal costs balance. Compute the minimum acceptable precision from the value of a hit and the cost of an attempt, and add the adoption cost that the formula omits. The base rate governs whether an accurate system is useful at all. Thresholds are governance: they allocate human attention and should be written down and revisited.

Worked example — the review budget. A merchant has 300 orders a month, and 3% are disputed. A model flags likely disputes for pre-emptive contact, which costs $2 and prevents $40 of loss in 30% of true disputes. With 3% prevalence among 300 orders there are 9 disputes. A perfect model saves $9 \times 0.3 \times \$40 = \$108$ for $18 of contact. A model with 60% recall and 50% precision contacts $9 \times 0.6 / 0.5 \approx 11$ orders, saving $9 \times 0.6 \times 0.3 \times \$40 = \$64.8$ for $22 of contact. The system is worth about $43 a month. At that value the correct decision may be not to build it — and knowing that before building is the entire benefit of doing the arithmetic.

Exercises. (recall) Write the expected-cost formula and define each term. (applied) Compute the minimum acceptable precision for a queue where a salesperson's hour is worth $20, an attempt takes fifteen minutes, and a conversation is worth $90; then recompute when a bad attempt costs $10 in reputational damage. (critical) "Most AI projects would be cancelled by a page of arithmetic." Argue with examples.

Further reading. Kahneman and Tversky (1973) on base rates; Gigerenzer et al. (2007) on communicating risk; [COMCEC], on why data quality and use-case definition precede technique.

PART THREE

The Machine

CHAPTER 13

Models in production

Learning outcomes. By the end of this chapter you should be able to: describe the components of a production inference path; explain the three failure modes unique to deployed models; design a monitoring plan; and state the difference between a model and a system.

13.1 A model is not a system

A trained model is a file. A system is a pipeline of components, any of which can fail: data acquisition, preprocessing, the model, post-processing, the store, the interface, and the person.

Sculley and colleagues' well-known paper on hidden technical debt makes the point structurally — that the model is a small box in a large diagram [Sculley et al., 2015]. The applied consequence is that most production incidents are not modelling incidents. They are schema changes, timeouts, rate limits, expired credentials, changed input distributions, and the quiet accumulation of glue code that nobody owns.

For a student, the lesson is to reason about the path a single request takes, and to ask at each step: what happens if this step fails? The answers are the system's design.

13.2 The inference path

Take a system that, given a business directory page, returns a structured record and a decision. The path has eight steps:

1. **Trigger.** A scheduled job, a person pressing a button, or an event.
2. **Fetch.** An HTTP request to the source. Failure modes: timeout, rate limit, block, changed markup, redirect to a login.
3. **Parse.** Convert the response into text or fields. Failure mode: a silent parse that returns an empty string, which is indistinguishable from a page that genuinely had no content unless you check.
4. **Normalise.** Strip boilerplate, collapse whitespace, repair mojibake, detect language. Failure mode: over-aggressive stripping that removes the content along with the navigation.
5. **Extract.** Apply the model or rules. Failure mode: confident nonsense; latency spikes; cost spikes if the call is per-item and the batch is large.
6. **Decide.** Apply the admission rule. Failure mode: a rule that admits everything, because a rule with no false rejections was written by someone optimising for recall.
7. **Persist.** Write to the store with provenance. Failure mode: a write that fails silently and a “success” reported anyway.
8. **Surface.** Put it in front of a person. Failure mode: so much output that the person stops looking.

The eighth step is the one engineers forget, and it is the one where most systems actually fail. An inference path that ends in a queue nobody reads has produced nothing.

13.3 Three failure modes unique to deployment

Distribution shift. The inputs change. Directory markup is redesigned, a market's phone formats change, a new spam pattern appears. The model's accuracy on its test set is a historical fact. Monitoring must compare the distribution of inputs in production to the training distribution — a drift statistic per feature — not merely the outputs.

Feedback loops. If the system's output influences the data used to train its successor, the loop can amplify its own errors. A prospect scorer that only ever gets feedback on the leads it chose will never learn about the ones it rejected. The remedy is deliberate exploration — hold back a random sample and measure what happens — which is a small, cheap discipline that almost nobody practises.

Silent degradation. A dependency changes and the system keeps returning results, now wrong. A language model's underlying version is updated; an API's default parameter changes; a currency's minor units change. Silent degradation is detected only by assertions on invariants: checks that a record's fields are non-empty, that phone numbers normalise to a valid national format, that totals reconcile. Invariants are the cheapest monitoring there is, and they are the first thing written in any system that has already been burned once.

13.4 The monitoring plan

Five signals, reviewed on a schedule.

Volume. Records in, records out, per day. A drop to zero is the most common incident and the easiest to detect.

Rejection rate. The fraction of candidates refused by the admission rule. A sudden fall means the rule has stopped discriminating; a sudden rise means the source changed.

Invariant failures. Counts by invariant. This is the signal that catches schema and format changes before a human sees a bad record.

Outcome rate. The downstream success measure — for a prospect system, the fraction of records that resulted in a conversation. This is the only signal that measures the system rather than its plumbing, and it moves slowest.

Cost per unit. Calls, tokens, bandwidth, storage. A system that becomes ten times more expensive is a system that will be switched off.

A monthly review of five numbers, written down, is sufficient for a small deployment. It is not sophisticated, and it is the difference between a system that is maintained and one that is abandoned.

13.5 Versioning and rollback

Three artefacts must be versioned together: the code, the model, and the data schema. If you cannot answer “which model produced this record, from which code, against which schema”, you cannot debug an incident or defend an outcome.

Rollback must be a single action. Fast rollback capability has a subtle design benefit: it lets you deploy more often, because the cost of a mistake is low. Systems without roll-

back are deployed rarely, and rare deployment is itself a risk, because each deployment is unfamiliar.

13.6 Summary

A model is a file; a system is a path with eight steps, each of which can fail. Deployment introduces distribution shift, feedback loops and silent degradation. Monitor five signals — volume, rejection rate, invariant failures, outcome rate, cost — and version code, model and schema together so that rollback is one action.

Worked example — the empty parse. A scheduled job reports success, and the queue is empty. What are the four most likely causes, in order of discovery cost? Answer: (1) the fetch returned a block page or a redirect, and the parser returned an empty string; (2) the parser's selector no longer matches because the markup changed; (3) the admission rule rejected everything because a field it depends on became null; (4) the persist step failed and the trigger ignored the error. Note that all four are silent, and all four would have been caught by a single invariant: a parse that produces zero characters from a non-empty response is an error, not a result.

Exercises. (recall) List the eight steps of an inference path and one failure for each. (applied) For a system you use, draw its inference path and mark the step where a failure would be least visible. (critical) "Monitoring is more valuable than model improvement." Discuss for a system with a 70% precision and a stable data source.

Further reading. Sculley et al. (2015); Breck et al. (2017) on the ML test score; [COMCEC], on deployment and results collection.

CHAPTER 14

Algorithms, procedures and pseudocode

Learning outcomes. By the end of this chapter you should be able to: distinguish an algorithm from a program; write pseudocode for a search and a sort; analyse the time complexity of a simple algorithm; and explain why algorithmic thinking precedes language choice.

14.1 The idea of an algorithm

NITI Aayog's Unit 5 develops problem solving and decision making through algorithms, searching and sorting, and presents pseudocode as the language in which a procedure is expressed before it is written in any particular programming language [NITI Aayog]. Unit 6 then introduces Scratch and Python as the means of execution [NITI Aayog]. The sequencing is exactly right, and worth defending: a student who learns a language first learns to think in that language's idioms, and mistakes the idioms for the ideas.

An **algorithm** is a finite, unambiguous sequence of steps that transforms an input into an output and terminates. Four properties to check:

- **Finiteness** — it stops.
- **Definiteness** — each step is unambiguous.
- **Input and output** — specified, with types.
- **Effectiveness** — each step is doable in principle.

A **program** is an algorithm plus everything else: data structures, error handling, memory, I/O, and the compromises of a real machine. This book's case is mostly the "everything else", and that is not a defect of the case; it is the ordinary condition of practice.

14.2 Searching

Linear search. Examine each element in turn until found. Pseudocode:

```
LINEAR-SEARCH(A, x)
  for i = 1 to length(A)
    if A[i] = x
      return i
  return NOT_FOUND
```

Cost: at most n comparisons, so O(n). Works on unsorted data. Correct and often the right answer for small n, which is most n in small business — a merchant with two hundred products does not need a binary search.

Binary search. Requires sorted data. Halve the interval each step:

```
BINARY-SEARCH(A, x)
  lo = 1; hi = length(A)
  while lo <= hi
    mid = floor((lo + hi) / 2)
    if A[mid] = x      return mid
    else if A[mid] < x lo = mid + 1
    else               hi = mid - 1
  return NOT_FOUND
```

Cost: O(log n). For a million records, about twenty comparisons rather than a million. The precondition — sortedness — is the whole lesson: the algorithm's efficiency is bought

with a maintenance obligation, because every insertion must preserve the order.

Hashing. Compute a key from the item and go directly to a bucket. Expected O(1) lookup. The trade is space and the requirement that the key function be stable. Most “search” in production software is this, and the reason a database index feels instant.

14.3 Sorting

Insertion sort. Keep a sorted prefix; insert each new element into place. $O(n^2)$, but excellent on small or nearly-sorted inputs, which is why real libraries use it below a threshold.

Merge sort. Divide, sort each half recursively, merge. O(n log n) guaranteed, stable, requires O(n) extra memory.

Quicksort. Choose a pivot, partition, recurse. O(n log n) in expectation, $O(n^2)$ worst case with poor pivots, in-place. The engineering lesson is that the expected case and the worst case can differ by orders of magnitude, and which one matters depends on whether an adversary controls the input.

14.4 Complexity, in the only form a student needs

Complexity analysis asks how run time grows with input size, ignoring constants. The common classes, in increasing order of cost: O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n), $O(n^2)$ quadratic, $O(2^n)$ exponential.

Two practical rules. First, **the constant matters at small n**. A quadratic algorithm with a tiny constant beats a linear algorithm with a large one for the sizes a small firm actually has. Second, **the growth rate matters at large n**.

The crossover is the only interesting question, and it is answered by measurement, not by assertion.

Complexity is also a modelling tool. If an approach to a problem is exponential in the input, no amount of hardware will rescue it, and the correct response is to change the problem — approximate, restrict, or use a heuristic. That move, from an intractable exact formulation to a tractable approximate one, is the same move that Chapter Ten makes when it relaxes a task definition until failure modes become visible.

14.5 Pseudocode as a contract

Pseudocode earns its place because it is inspectable by people who cannot read the implementation language — which is to say, by the person who owns the business. A one-page pseudocode description of the admission rule is a document a shopkeeper can argue with. A two-hundred-line implementation is not.

This is a design principle with practical consequences. In the case, the admission rule is described as seven predicates, each of which a non-programmer can evaluate on a printed page: does the page name a business, is there a street address, is there a phone number, is the number plausible for the country, is the business in an excluded class, has it been contacted before, is it on the do-not-contact list. The predicate form is not merely readable; it is auditable, because each predicate can be tested independently against a sample.

14.6 Summary

An algorithm is a finite, unambiguous, terminating procedure; a program is an algorithm plus the compromises of a

machine. Binary search buys O(log n) with a sortedness obligation; hashing buys O(1) with space; sorting algorithms trade worst-case guarantees for memory or in-place behaviour. Complexity analysis identifies which approaches cannot be rescued by hardware. Pseudocode is a contract readable by the person who owns the consequences.

Worked example — choosing a search. A merchant has 4,000 product rows in a spreadsheet and wants instant search in a browser. Which structure? Answer: neither binary search nor a database. Load the 4,000 rows once, build an inverted index from terms to row IDs in memory, and intersect posting lists per query. The linear pass over 4,000 rows would take milliseconds anyway; the index makes it microseconds and, more importantly, supports multi-term queries with ranking. The lesson is that the data structure determines what query is cheap, and 4,000 rows is small enough that the choice is about features, not speed.

Exercises. (recall) State the precondition of binary search and the complexity of each algorithm in §14.3. (applied) Write pseudocode for a procedure that takes a list of orders and returns those unpaid for more than thirty days, and state its complexity. (critical) “Teaching pseudocode before a language produces students who cannot ship.” Argue both sides.

Further reading. [NITI Aayog], Units 5 and 6; Cormen et al. (2022), chapters 1–2; Knuth (1997), chapter 1.

CHAPTER 15

Languages, APIs and the cloud

Learning outcomes. By the end of this chapter you should be able to: describe the role of a programming language and of a library; explain what an API is and how a request is structured; classify cloud service models; and estimate the cost of a small deployment.

15.1 Languages

Microsoft's module introduces R and Python as the languages of data work [Microsoft], and the pairing is accurate: R for statistics and visualisation, Python for general-purpose engineering with a data ecosystem. NITI Aayog introduces Scratch for beginners and Python for continuation [NITI Aayog], which reflects a defensible pedagogy — visual blocks to establish control flow and decomposition, then text to establish precision.

What a language actually provides, in order of importance: a way to express control flow; data structures; a standard library; and an ecosystem of third-party packages. The last is decisive in practice. Python's dominance in applied AI is not a property of the language's syntax; it is a property of the packages available in it — numeric arrays, data frames, plotting, and the client libraries for every model provider.

The choice of language is therefore mostly a choice of ecosystem, and the ecosystem is mostly a choice of what you will not have to write yourself. A student should learn one language well enough to debug, and should learn to read

the documentation of libraries, which is the actual daily skill.

15.2 What an API is

An **application programming interface** is a contract: a defined way for one program to ask another for something. In web systems, the contract is usually HTTP plus JSON, with four elements.

Endpoint and method. A URL and a verb: `POST /v1/messages` means "create a message at this resource".

Headers. Metadata: content type, authentication, a request identifier. Authentication is typically a key or token in a header, which is why keys must never be committed to source control — a lesson this book's author learned in the ordinary way, by having to rotate them.

Body. The payload, usually JSON.

Status code and response. 200 for success, 4xx for a client error, 5xx for a server error, plus a JSON body. The distinction between 4xx and 5xx is operational, not pedantic: a 4xx means do not retry, fix the request; a 5xx means retry with backoff.

Microsoft's module describes how the major cloud platforms expose AI capabilities as APIs grouped into families — vision, speech, language, search, and decision — with named services inside each family, such as image analysis, face, speech-to-text, translation, language understanding and question answering [Microsoft]. The grouping is useful because it maps capability to contract: the family tells you the shape of the input (an image, an audio stream, text, a query) and the shape of the output.

15.3 Conversational AI as a case study in contracts

The Microsoft module's treatment of conversational AI is the clearest worked example of the whole idea. A conversational system, it explains, depends on two mechanisms: **intent classification** — deciding what the user wants — and **entity extraction** — pulling the specific details out of the utterance [Microsoft].

Those two mechanisms are two API contracts, and separating them is a design insight. "Book a taxi to the airport at six" contains an intent (book a taxi) and entities (destination: airport; time: 06:00). A system that only classifies intent can route the request but cannot act on it; a system that only extracts entities knows the details but not what to do. Production systems chain them, and the failure modes are distinct: intent errors produce the wrong workflow, entity errors produce the right workflow with wrong data — and the second is more dangerous, because the user sees a confirmation for the wrong booking rather than an error.

That asymmetry is the same asymmetry as Chapter Twelve's settlement example: a visible mistake is cheaper than a confident one.

15.4 The cloud

Microsoft's module distinguishes **private**, **public** and **hybrid** clouds [Microsoft]. The distinction is about ownership and access, and it has three practical consequences for a small firm.

Cost structure. Public cloud converts capital expenditure into operating expenditure and scales to zero. For a small

firm this is decisive: a system that costs nothing when idle can be built before it has revenue.

Operational burden. Private cloud requires someone to run it. A firm of one cannot, which is why the case runs on a public platform's free tier.

Data governance. Where data resides determines which law applies to it. A merchant's customer records may not lawfully leave a jurisdiction, and the cloud choice is therefore a legal decision before it is a technical one.

The free tier deserves specific mention because it is the actual enabling condition for a class of systems. A serverless function platform with a monthly request allowance, a key-value store with a storage allowance, and an AI provider with a token allowance can together host a small commercial system at zero marginal cost. The design constraint that follows is severe and useful: **every architectural decision must fit inside allowances that are measured in requests, not in money.** Chapter Sixteen shows what that does to a quota design, and why the resulting system is more robust than one that simply pays for capacity.

15.5 Cost arithmetic

Three lines, computed before building.

Compute. Requests per month × requests per user action × cost per request. On a free tier, this is zero until the ceiling, then it is a cliff — which must be known in advance, because a cliff at 100,000 requests is a system that works until the day it suddenly does not.

Storage. Records × bytes per record. Key-value stores are cheap per gigabyte and expensive per read, which inverts the usual database intuition and pushes design toward fewer, larger reads.

Model calls. Calls × tokens per call × price per token. This is the line that surprises people, because a per-call cost that looks trivial becomes the dominant cost when a loop multiplies it. The mitigation is architectural: cache, batch, and cap.

15.6 Summary

A language is mostly its ecosystem; learn one well and learn to read documentation. An API is a contract of endpoint, headers, body and status; 4xx means fix, 5xx means retry. Conversational systems chain intent classification and entity extraction, and entity errors are more dangerous than intent errors. The cloud's categories are about ownership, cost structure and jurisdiction, and the free tier is an enabling condition with a cliff that must be mapped in advance.

Worked example — pricing a feature. A feature summarises each new order with one model call of about 800 tokens. At 300 orders a month, and a price of $0.0004 per thousand tokens, the monthly cost is 300 × 800 × 0.0000004 = $0.096. Now suppose the feature is also applied to all historical orders on first load: 12,000 orders × 800 tokens = $3.84 one-off. Both are trivial. Now suppose the summary is regenerated every time the order page is viewed, at 40 views a day: 40 × 30 × 800 × 0.0000004 = $0.38 a month — still trivial, but the architecture is now wrong, because a read has become a model call and the

cost scales with engagement rather than with orders. Cache it.

Exercises. (recall) Name the four parts of an HTTP request and what each carries. (applied) Estimate the monthly cost of a system that makes 50,000 API calls, each returning 2 KB, with a model call on 10% of them. (critical) “Free tiers are not free; they are a subsidy that ends.” Discuss with reference to what a business should do when the subsidy ends.

Further reading. [Microsoft], “Cloud Computing” and “APIs”; [NITI Aayog], Unit 6; Fielding (2000), chapter 5.

PART FOUR

The Case

CHAPTER 16

Designing under constraint: the rail

Learning outcomes. By the end of this chapter you should be able to: explain how a hard resource ceiling changes a system's architecture; describe a quota reserve ladder and why it protects the primary transaction; justify a no-custody design; and explain what makes a statement recomputable.

16.1 The system, described plainly

Tapstall is a commerce rail for small merchants in Zimbabwe, South Africa, Kenya and Nigeria. A merchant gets a storefront at a link, a product catalogue, an order flow in which the buyer confirms with one tap, and a settlement statement reconciling orders against payments the merchant received directly. The merchant pays a flat fee. The platform never touches the money.

The implementation is a single serverless worker on a public cloud's free tier, with a key-value store, and no other persistent infrastructure. At the version this chapter describes, the code is roughly eleven thousand lines across four modules — the main worker, a core of shared logic, a discovery module, and a settlement rail — and it carries a test suite of four hundred and nineteen assertions. The monthly infrastructure cost is zero.

Everything interesting about the architecture follows from three constraints: the cost ceiling, the absence of custody, and the fact that the operator is one person.

16.2 The quota ladder

A free tier is not infinite capacity; it is a monthly allowance that becomes a hard stop. The platform's allowances at this version were 100,000 reads a day, 100,000 writes a day, 1,000 each of three further operation classes, and one gigabyte of storage. The five limits are not equivalent: two are large and three are small, and the small ones are the binding constraint. A system designed against the large numbers will hit a wall on the small ones.

The naive design is to count requests and refuse when the count reaches the limit. It fails for two reasons. First, a counter shared by all operations lets a non-critical operation starve a critical one — a burst of catalogue browsing can exhaust the allowance that order submission needs. Second, a counter that is checked before use and decremented after is not atomic, so concurrent requests can all pass the check and together exceed the limit.

The design that replaces it is a **reserve ladder**: a single counter is checked against a descending series of thresholds, and operations of different criticality require different thresholds to proceed. In the deployed system the ladder is 1000 / 975 / 900 / 800 / 700 against a nominal daily budget of 1000 units:

Threshold	What may proceed	Rationale
1000	Background and analytic work	First to be refused as pressure rises
975	Discovery and prospecting	Expensive, deferrable
900	Catalogue and storefront reads	Degradable
800	Non-essential writes	Deferrable
700	Order submission only	Protected to the last reserve

Read the ladder as a statement of priorities made executable. At 700 units remaining, the system will serve orders and nothing else. Its behaviour under pressure is not an emergent property of load; it is a decision taken in advance and encoded.

The ladder has a property worth naming because it is the reason to prefer it to a plain cap: **starvation freedom**. Because order submission is admitted down to the lowest threshold and the lowest threshold is the last to be reached, an order submitted at any point in the day, in any load condition short of exhaustion of the entire budget, will be admitted. The proof is one line — a resource that is reserved to a class cannot be consumed by another class — and its practical value is that the merchant experience is stable even when the platform is under strain.

16.3 Why no custody

The platform holds no money. Buyers pay merchants directly — by mobile money, bank transfer, cash, or whatever the merchant already uses — and the platform records

that a payment was claimed and whether it was confirmed. The design decision is not a limitation reluctantly accepted; it is the source of the system's regulatory, financial and moral simplicity.

Four consequences.

No licensing burden. Holding customer funds is a regulated activity in every jurisdiction in which the platform operates. Not holding them removes an entire category of legal obligation, capital requirement and audit.

No float, no incentive to delay. A platform holding funds has a reason, however innocent, for the money to move slowly. A platform that never holds funds cannot develop that incentive, and the merchant never has to wonder.

No counterparty risk. If the platform fails, the merchants have their money, because the platform never had it. The statement is a record, not a claim on an asset.

A different failure mode. The system can be wrong about what happened but cannot be wrong about where the money is. That is a much better failure mode, and Chapter Twelve's asymmetry analysis is the reason: the error that remains is visible and correctable, and the error that would have been catastrophic is structurally impossible.

The rail's six gates all enforce this: an order cannot transition to a paid state without an explicit claim and a confirmation, and no code path creates a balance that could be withdrawn.

16.4 The recomputable statement

The artefact that carries the merchant relationship is a settlement statement: a document listing the period's orders, the payments claimed, the payments confirmed, and the resulting position — plus the platform fee, computed as a flat charge rather than a percentage.

Two design properties make it trustworthy.

Recomputability. Every figure on the statement can be derived from the underlying order records by a stated rule. Nothing is stored as a total that cannot be re-derived. If the statement says a merchant is owed a particular figure, a person with the order list and the rule can arrive at the same figure independently — and if they cannot, the discrepancy is itself informative.

Immutability of recorded facts. Once an order is recorded, its identity does not change; corrections are appended as new events, so the statement for a past period, regenerated, produces the same numbers. A statement that changes retroactively is not evidence.

The flat fee matters for the same family of reasons. A percentage fee makes the platform's revenue a function of the merchant's volume, which creates an incentive to maximise volume and a tendency to describe the fee as a partnership. A flat fee makes the platform's revenue independent of the merchant's success, which is honest about what is being sold: software, not a share of a business. It also makes the fee predictable, which for a small merchant is worth more than a marginally lower rate with a variable total.

16.5 Design as restriction

The pattern across the rail is that the good decisions are restrictions: a ladder that refuses work, a rule that forbids custody, a fee that does not scale, a statement that can be recomputed rather than trusted.

This is the opposite of the usual engineering instinct, which is to add capability. The instinct is right in an environment with slack — with money, staff and time — because capability is cheap there and failure is contained. In an environment where one person operates the system and the cost ceiling is hard, capability is a liability and every added feature is a new failure mode that nobody will have time to notice.

A student should be able to state the general principle: **the constraints of the operator determine the correct architecture more than the requirements of the user do.** Two firms with identical users and different staffing should build different systems. Chapter Five said this in the abstract; the rail is the worked instance.

16.6 Summary

Three constraints — cost ceiling, no custody, one operator — determine the rail's architecture. The quota ladder turns priorities into executable policy and guarantees that order submission is the last thing refused. No custody removes a category of legal and financial risk and replaces a catastrophic failure mode with a visible one. The statement is recomputable from order records, so trust is a property of the arithmetic rather than of the vendor.

Worked example — placing a new feature on the ladder. A feature would read the catalogue on every store-

front visit to personalise recommendations, at an estimated 30,000 reads a day. Where does it go? Answer: it is a reader on the critical path of a visitor, but not on the critical path of an order. On a 100,000-read allowance, 30,000 is 30% of the daily budget consumed by a non-essential feature — it must sit at a high threshold, be cached aggressively, and be disabled entirely when the remaining budget falls below 900. If it cannot be built to run below those thresholds, it should not be built.

Exercises. (recall) State the five thresholds of the ladder and the operation admitted at each. (applied) Design a reserve ladder for an imaginary system with three operation classes and a budget of 500 units, and justify the ordering. (critical) “A flat fee is an ethical position, not a pricing strategy.” Discuss.

Further reading. Chapter Six of this book’s companion volume; the author’s paper on order-without-custody architecture (Garikai, 2026); Fielding (2000) on statelessness.

CHAPTER 17

The gate: finding prospects without lying to yourself

Learning outcomes. By the end of this chapter you should be able to: explain what an admission rule is and why it belongs in a discovery system; evaluate a multi-predicate gate; normalise a phone number; and interpret a result of zero prospects from forty-seven records.

17.1 The problem

A new merchant platform has no customers. The first hundred merchants must be found, and there is no budget for advertising. The available material is public: business directories, chamber-of-commerce listings, trade association pages. The task is to convert those pages into a list of businesses a salesperson can call, with a reason to call.

The obvious implementation is a scraper plus a language model: fetch pages, ask a model whether the page describes a business, extract a name and a phone number, write to a spreadsheet. This implementation produces a list quickly, and the list is worthless, for reasons that the base-rate arithmetic of Chapter Twelve predicts. Most pages that mention a business are not the business's page. Directory indexes, category pages, news articles, blog posts, and aggregator listings all mention many businesses. A model asked "is this about a business?" answers yes, correctly, and the record is still unusable, because the question was wrong.

17.2 The admission rule

The fix is to replace the question with a set of predicates, each of which must hold, and each of which corresponds to a fact that makes the record usable. In the deployed system there are seven.

1. **The page identifies a business** — a trading name, present as a heading or a structured field rather than as a passing mention.
2. **A street address is present** — not merely a city, because a city is not a location for a visit and does not distinguish a shop from a listing.
3. **A telephone number is present.**
4. **The number is dialable** — it normalises to a valid national format for the stated country (see §17.4).
5. **The business is not in an excluded class** — charities, public bodies, competitors, and any entity whose inclusion would be a harm rather than a waste.
6. **It has not been contacted before** — the do-not-contact record and the contact history are checked before admission, not after.
7. **It is on no do-not-contact list** — a distinct check from six, because a business may ask never to be contacted without ever having been contacted.

Predicates five to seven are not quality filters. They are ethical constraints expressed as code, and their position in the gate — before a record exists rather than after — is the point. A record that should not be contacted is never created, so no downstream process can contact it by mistake. This is cheaper and more reliable than filtering at

send time, and it means the exclusion cannot be lost in a hand-off between systems.

17.3 The result of the first run

The first full run of the gate against a directory harvest produced forty-seven records and zero admitted prospects.

Zero, from forty-seven. A student should read that result as information rather than failure, and the correct next step is to examine the rejection reasons rather than to loosen the gate. The reasons fell into three groups.

Pages that were listings, not businesses. Category pages and index pages that mention dozens of names, none of them with an address in the page body.

Businesses with no usable telephone. A name and a physical description but no number anywhere on the page, or a number belonging to the directory rather than the business.

Numbers that did not normalise. Well-formed to the eye, unusable to a dialler.

Two things follow. First, the harvest was pointed at the wrong kind of page: the source was an index, and the gate correctly refused an index. Second, and more important, the gate had done its job — it had converted a plausible-looking list into an accurate statement that this source, harvested this way, yields nothing usable. That statement is worth more than a list of forty-seven bad records, because it redirects the work rather than hiding the problem inside it.

The general principle: **a filter's output rate is a measurement of the source, and should be read as one.** A

gate that admits everything tells you nothing about the data; a gate that admits nothing has located the flaw precisely.

17.4 The number that could not be dialled

The fourth predicate deserves its own section because it is the smallest and most instructive component in the system.

Nigerian numbers are frequently written with a trunk prefix in brackets after the country code — the form `+234 (0)730 177 1710`. The bracketed zero is a domestic convention: within Nigeria one dials `0730…`, and internationally one drops the zero. The string `+234 (0)730 177 1710` therefore denotes a valid number, and a naive normaliser that strips spaces and brackets produces `+23407301771710` — which has one digit too many and is not dialable. A model asked to extract "the phone number" will return exactly that string, because it is, character for character, what the page says.

The correct normalisation strips the trunk prefix when a country code is present: `+234 (0)730 177 1710` → `23407301771710` becomes `2347301771710`. The rule is small, the failure is total — every Nigerian record fails, and it fails at the last step, after all the expensive work has been done.

Three lessons, each generalisable.

Format validity is not dialability. A string can be well-formed and unusable. Validation must test the property the user needs, not the property that is easy to check.

A model asked to extract returns what is written. Extraction and normalisation are different tasks, and a stat-

istical system given the first will not perform the second. The pipeline needs an explicit normalisation stage with country-specific rules.

The failure is silent. Nothing in the system reports an error; the numbers are simply unusable, and the discovery of the problem depends on someone trying to dial one.

17.5 Why discovery is bounded

The system has a hard daily ceiling on new records — twenty, in the deployed configuration — and the ceiling is not a cost control. It is an absorption control, in the sense of Chapter Four's third failure pattern and Chapter Five's third constraint.

A salesperson can work a bounded number of records per day. A system that produces more than that bound produces a backlog, and a backlog converts a queue of opportunities into a queue of obligations. Once a queue is an obligation, it is abandoned, and once abandoned it is never reopened.

The ceiling also protects the source. Twenty requests a day is a rate that no directory operator will notice or resent; a thousand is a rate that gets an address blocked, and the block is a permanent loss of the source. Bounded politeness is a technical survival strategy.

17.6 Summary

Discovery fails when the question is wrong: "is this about a business?" admits everything and yields nothing usable. A gate of seven predicates, three of which are ethical constraints, converts the question into facts that make a record actionable, and excludes inadmissible entities before

they exist. A zero-admission run is a measurement of the source. Phone normalisation must test dialability, not format, because extraction returns what is written. The daily ceiling is an absorption control and a politeness control, not a cost control.

Worked example — diagnose the gate. A new source yields 300 records, of which 240 are admitted. What do you check, and in what order? Answer: (1) sample ten admitted records and try to dial them — if the numbers fail, predicate four is broken; (2) check the rejection reasons for the 60 refused, which should be plural; if they are homogeneous, the source is homogeneous and the gate is not discriminating; (3) look for excluded classes among the admitted — if charities appear, predicate five is not wired; (4) check the address field for city-only values. An 80% admission rate on a directory is implausible and is more likely a broken gate than a clean source.

Exercises. (recall) List the seven predicates and say which are quality filters and which are ethical constraints. (applied) Write a normaliser in pseudocode for your own country's phone formats, including the trunk-prefix case, and list five strings it must handle. (critical) "A gate that admits nothing is a success." Defend, then state the condition under which it is a failure.

Further reading. [Microsoft], "Conversational AI" on intent and entities; the author's release history for version 1.7.x; RFC 3966 on telephone URIs.

CHAPTER 18

What broke

Learning outcomes. By the end of this chapter you should be able to: classify production failures into the five recurring classes; explain why each class is invisible to the test suite that missed it; and design a defence for each.

18.1 The five classes

Across the platform's history, failures cluster into five classes. The taxonomy is empirical — it was assembled by reviewing what actually went wrong — and it is useful because each class has a different defence.

Class 1: the empty success. An operation reports success and produces nothing. The parse that returns an empty string, the write that silently fails, the send that is queued and never delivered. Invisible because the return code is correct.

Class 2: the wrong unit. A correct computation on the wrong quantity. Money in cents treated as dollars; a rate per day applied per hour; a period inclusive at both ends and therefore one day too long. Invisible because the arithmetic is right.

Class 3: the format that is not the thing. A value that satisfies every structural check and is unusable — the undialable number, the address that is a city, the product code that exists in no catalogue. Invisible because validation tests shape, not meaning.

Class 4: the rule that admits everything. A filter with no false rejections, because it was tuned until nothing was

rejected. Invisible because the absence of rejections looks like quality.

Class 5: the work nobody can absorb. Functionally correct output at a volume the human process cannot consume. Invisible because the metric improves.

Each class is invisible to a different kind of test, and that is why they persist. A unit test with a mocked dependency catches Class 1 only if the mock models emptiness; a test of the arithmetic catches nothing in Class 2 if the units were wrong in the fixture; a schema validation catches nothing in Class 3; a test of the filter catches nothing in Class 4 if the test set contains only positives; and no test at all catches Class 5, because Class 5 is a property of the organisation, not the code.

18.2 The four hundred and nineteen tests

The test suite is worth describing because its shape is a response to the taxonomy. The four hundred and nineteen assertions are distributed across the modules roughly in proportion to their risk, and their most interesting property is what they pin.

Pin the invariants, not the outputs. A test that asserts a statement's total equals a stored number is fragile and weak; a test that asserts the total equals the sum of the lines, recomputed, is a statement about the system's structure. The suite prefers the second.

Pin the refusals. Tests that assert an order cannot reach a paid state without a claim and a confirmation; tests that assert a do-not-contact business cannot be admitted; tests that assert the quota ladder refuses the correct class at

each threshold. These tests fail if someone adds capability, which is exactly when you want a test to fail.

Pin the shapes the outside world depends on. Field names, response shapes, and the CSV columns a merchant's spreadsheet expects. A rename is a breaking change to a person's workflow, and the test is the only thing that remembers.

What the suite cannot do is catch Class 5, and this is worth stating plainly to students, because the temptation is to believe that a large suite means a safe system. It does not. The suite covers what has been thought of; the failures that matter most are the ones nobody thought of, which is why production monitoring (Chapter Thirteen) is not redundant with testing.

18.3 Three incidents, in detail

The statement that changed. A merchant viewed a past period's statement twice and saw different figures. The cause was a recalculation that included orders recorded after the period end — a boundary condition, an off-by-one in the date range, Class 2. The defence adopted was structural rather than corrective: period boundaries are computed once and stored with the statement, and the recomputation is asserted to be idempotent. The lesson is that a defect in a financial document is not fixed by fixing the defect; it is fixed by removing the class of defect from the design.

The queue that filled with page titles. Early discovery admitted records whose only business-like property was a name in a heading. The gate's predicates were added in response, one at a time, each after a specific bad record.

This is the ordinary way admission rules are built, and it argues against designing them up front: you cannot enumerate the failure modes of a source you have not read.

The copy that read as a threat. A generated promotional message, reviewed by a human, still contained a construction that a recipient could reasonably read as pressure — manufactured urgency about a deadline that did not exist. The response was a doctrine (Chapter Nineteen) and a review rule, not a filter, because the failure was semantic and no regex catches it.

18.4 Defence in the design, not the test

The general lesson across the five classes is that each has a structural defence, and the structural defence is usually cheaper than the test that would catch the same failure.

Class	Structural defence
Empty success	An invariant: zero output from non-empty input is an error
Wrong unit	Name every quantity with its unit, store money in minor units, and never convert implicitly
Format not thing	Validate the property the user needs, not the shape of the string
Admits everything	Require the filter to reject a known-bad sample as part of the test
Unabsorbable work	A hard ceiling on throughput, expressed as a constant with a comment explaining the human constraint

None of these is sophisticated. All of them are cheap. The reason they are rare is that they require somebody to decide, in advance, what the system is not allowed to do, and

that decision is uncomfortable — it forecloses options and it can be wrong.

18.5 Honesty about the commercial case

This chapter would be dishonest if it stopped at engineering. At the time of writing, Tapstall has no paying customers. The architecture is sound, the test suite passes, the statement is recomputable, the gate refuses correctly — and nobody has bought it.

A student should sit with that, because it is the most useful fact in the case. **Software quality and commercial success are different variables**, weakly coupled at best. The platform is good at the things this book can teach and unproven at the thing it cannot: whether a sufficient number of merchants will pay a flat fee for a storefront and a statement. That question is answered by selling, not by building, and the honest position is that it remains open.

The engineering consequence is a discipline worth naming: build so that the unproven part can be tested cheaply. The free tier means the system costs nothing while dormant; the flat fee means a single customer is a meaningful experiment; the statement means the product can be shown in one screen. A system designed to survive its own uncertainty is a better system than one designed for a success that has not happened.

18.6 Summary

Production failures fall into five classes: the empty success, the wrong unit, the format that is not the thing, the rule that admits everything, and the work nobody can absorb. Each is invisible to a different test, and each has a structural defence cheaper than the test. A large suite is

not a safe system. And software quality is not commercial success — a distinction the case makes unavoidable.

Worked example — classify these failures. (a) A merchant's monthly total is correct except in months with thirty-one days. (b) A prospect list contains a company that asked never to be contacted. (c) A generated description is accurate but 400 words long, and the merchant posts none of them. Answer: (a) Class 2, boundary/unit; defence is a period computation stored once and an idempotence test. (b) Class 4 and a broken predicate; defence is exclusion at admission, not at send. (c) Class 5; defence is a length ceiling and a one-line default.

Exercises. (recall) Name the five classes and the defence for each. (applied) Take a system you have built or used and find one instance of each class in its history. (critical) "A test suite that never fails on a capability addition is a test suite that pins nothing." Discuss.

Further reading. Sculley et al. (2015); Nygard (2018) on failure and stability patterns; the author's release log for versions 1.7 to 1.9.

PART FIVE

Judgement

CHAPTER 19

Ethics, consent and the things a system should refuse

Learning outcomes. By the end of this chapter you should be able to: identify the four ethical questions that arise in any data-collecting system; apply a consent test to a data collection proposal; distinguish fairness as a metric from fairness as a procedure; and state rules a system should enforce against its own operator.

19.1 The scenarios that teach it

NITI Aayog's Unit 7 on ethics does something that most treatments avoid: it puts the student inside a decision. The module presents scenarios in which data is to be collected from people and asks what should happen — whether consent is required, what happens when someone refuses, whether the person asking should force the answer, and how to handle collection where the point is that no individual can be identified [NITI Aayog].

The pedagogy is right because ethics is not a list of principles; it is a set of decisions made under pressure, usually by someone junior, usually with a deadline. A student who has rehearsed the decision in a classroom makes it better in an office.

Four questions recur in every system that collects data about people.

Who decided that this collection would happen, and do they have the standing to decide? Consent obtained by a salesperson from a shopkeeper who does not under-

stand what is being collected is not consent; it is a signature. The test is whether the person could refuse without cost and could explain the collection to someone else afterwards.

What happens to the person who declines? If refusal means exclusion from a service they need, the choice is not free and the consent is nominal. Systems that require data as the price of participation should say so plainly rather than describing the arrangement as consent.

Is the answer being forced? The module's example asks, in effect, whether one should press a person to answer a question they have declined. The answer is no, and the design translation is a default of unknown rather than a default of no, because a default of no records a refusal as a fact about the person when the fact is about the encounter.

Can the person be identified, and does it matter? Anonymisation is a spectrum, not a binary. Removing a name from a record that contains a phone number, a street address and a transaction history has removed nothing. The module's treatment of anonymous collection makes the point that anonymity is a property of the whole dataset, not of a column.

19.2 Bias, in its operational form

The word "bias" carries three meanings in this literature, and conflating them makes the discussion useless.

Statistical bias is a systematic error in an estimate — a sampling procedure that over-represents part of the population. It is measurable and correctable.

Algorithmic bias is a systematic disadvantage in a system's outputs for a group, arising from the data, the label definition, or the objective. It is measurable in principle and often not measured.

Human bias is prejudice in the people who build, deploy and interpret the system. It is not a property of the model at all, and it is the most consequential of the three, because it determines the other two: what data is collected, how the task is defined, what counts as an error, and whose error is noticed.

The operational lesson is that fairness is a procedure before it is a metric. The procedures that matter are: define the task and the label in writing; measure performance by group; ask who is absent from the data; and give someone who is affected a way to contest an outcome. A system with no contest mechanism has no fairness regardless of its metrics.

In the case, the group-level question is concrete. A prospect gate trained on directory pages in one market will, applied to another, inherit that market's coverage: businesses without a website are absent, businesses in the informal sector are absent, and businesses whose names are spelled inconsistently are harder to de-duplicate. None of that is malicious, and all of it determines who receives a call — and therefore who gets access to the software. Coverage is an ethical question because opportunity follows it.

19.3 Transparency: who needs to know what

Transparency is not one thing. Three audiences need three different disclosures.

The affected person needs to know that a system is involved, what it decided about them, and how to contest it. Not the architecture; the decision and the appeal.

The operator needs to know the system's failure modes, its operating threshold, and what it does when a dependency fails. This is the audience that the present book serves.

The auditor or regulator needs provenance: which data, which version, which rule, which date. This is why Chapter Six's insistence on provenance at collection is not pedantry — a system that cannot answer "where did this value come from" cannot be audited, and a system that cannot be audited cannot be defended.

A system that gives the third disclosure but not the first is opaque in the way that matters, because the person with the least power is the one who cannot see the decision.

19.4 Rules a system enforces against its own operator

The most useful ethical architecture in a small system is a set of prohibitions the software enforces on the person running it. This is counter-intuitive — the operator is the owner — and it is exactly why it works: in the moment of trying to make a number, a person will do things they would not endorse in a calm review. A rule in the code is a decision made once, well, instead of repeatedly, badly.

Tapstall's promotional doctrine is such a set, and it has five standing prohibitions.

No manufactured scarcity. If stock is not limited, the copy does not say "only a few left". Urgency that does not exist is a lie with a conversion rate.

No invented social proof. No customer counts that are not real, no testimonials that were not given, no ratings that were not received. A new merchant has no social proof and the honest response is to sell on the product, not to simulate a crowd.

Shame no buyer. No copy that implies a person is foolish, poor, or late for not buying. The Emotion Doctrine permits urgency about a real deadline and forbids contempt for the reader.

No promises about outcomes. "Will increase your sales by 30%" is a fabricated statistic. What can be said is what the software does.

Never contact the merchant's customers. A platform that markets to its merchants' buyers has converted a tool into a competitor. This one is enforced by an architectural boundary, not a policy, which is why it holds.

The doctrine also has a positive form, which is the harder half: every message should give the recipient something before it asks for anything, in a ratio of roughly two useful communications to one promotional one. A ratio is a crude instrument, and it is better than the alternative, which is a norm of pure promotion that nobody can sustain without becoming a nuisance.

19.5 The ethical content of architecture

The chapter's argument is that in a small system, ethics lives in architecture more than in policy.

A policy says "we will not contact people who have asked us not to". An architecture excludes them at admission, be-

fore a record exists, so that no policy failure downstream can produce a contact.

A policy says “we will not hold customer funds”. An architecture has no balance to hold, so custody is not a discipline but an impossibility.

A policy says “we will not deploy manipulative copy”. An architecture has five named prohibitions and a review step, so the prohibition is a checklist item rather than a matter of the author’s mood on a Thursday.

The general form: **convert every ethical commitment you can into a structural constraint, and reserve policy for what cannot be structured.** The residue is real — semantic judgement about whether a sentence is coercive cannot be automated — and naming the residue honestly is more useful than claiming that policy covers it.

19.6 Summary

Ethics is a set of decisions under pressure, best taught through scenarios: who consented, what happens on refusal, is the answer forced, can the person be identified. Statistical, algorithmic and human bias are distinct, and fairness is a procedure before it is a metric. Three audiences need three disclosures, and the affected person’s is the one most often missing. Rules that a system enforces against its own operator are more reliable than resolutions, and every ethical commitment should be converted into a structural constraint wherever possible.

Worked example — audit a collection plan. A team proposes to photograph shopfronts in a city and use the images to infer product categories, storing the images with GPS coordinates. Apply the four questions. Answer:

standing — who decided, and is photographing a shopfront with people in it lawful in this jurisdiction? decline — a shopkeeper who objects is excluded from a service that may benefit them, so the arrangement must be opt-in. forcing — the photographer must be instructed not to press an objector, and to record refusal as unknown. identification — GPS plus a photograph identifies the premises and often the people in it, so the dataset is not anonymous and the retention period must be stated. The likely conclusion is that the plan is viable only with consent, blurring, a short retention period and a stated purpose.

Exercises. (recall) State the four ethical questions and one design response to each. (applied) Write five prohibitions for a system in your own domain, in the form of rules a junior colleague could apply without asking you. (critical) “Anonymised data is a fiction.” Discuss with reference to the case’s prospect records.

Further reading. [NITI Aayog], Unit 7, “Ethics”; Floridi and Cowls (2019); [COMCEC], on the limits of AI.

CHAPTER 20

Law, data protection and the state

Learning outcomes. By the end of this chapter you should be able to: name the data-protection principles common to the major regimes; explain the lawful-basis requirement; describe the obligations that fall on a small operator; and identify the three legal questions to resolve before collecting personal data.

20.1 The regimes that matter here

A student working in southern Africa operates under several overlapping regimes, and the good news is that their principles converge even where their details differ.

Zimbabwe. The Cyber and Data Protection Act (2021) establishes a data protection authority and requires lawful, fair processing of personal data, with obligations on data controllers to secure data and to respect the rights of data subjects.

South Africa. The Protection of Personal Information Act (2013, in force from 2021) sets eight conditions for lawful processing, including accountability, purpose specification, further processing limitation, information quality, openness, security safeguards, and the rights of data subjects.

Kenya. The Data Protection Act (2019) establishes similar principles and an office of the Data Protection Commissioner.

Nigeria. The Nigeria Data Protection Act (2023) establishes a commission and a framework of principles and lawful bases.

Europe. The General Data Protection Regulation (2016/679) is the most widely copied template: lawfulness, fairness and transparency; purpose limitation; data minimisation; accuracy; storage limitation; integrity and confidentiality; accountability.

The convergence matters practically. A small operator in Harare who designs to the GDPR's principles will be broadly compliant with the Zimbabwean, South African, Kenyan and Nigerian frameworks, because they share the same ancestry. Designing to the strictest applicable regime is the only sane strategy for a system that operates in several countries, and it is cheaper than building per-country variants.

20.2 The principles, translated into engineering

Each principle has a concrete form, and the concrete form is what a small team can act on.

Lawfulness and lawful basis. Every processing activity must have a legal basis: consent, contract, legal obligation, vital interest, public task, or legitimate interest. For a merchant platform, the basis for order data is contract; for marketing email, it is consent; for prospect discovery from public pages, it is the least settled, and the honest position is that business contact data published by a business for the purpose of being contacted is a weaker claim to legitimacy than consent, not a licence to do anything.

Purpose limitation. Data collected for one purpose is not free for another. Order data collected to fulfil an order cannot be repurposed for unrelated marketing without a new basis. The engineering form is a `purpose` field on every col-

lection, and a rule that a new use requires a new collection or a new justification.

Data minimisation. Collect what the purpose requires and no more. The engineering form is the admission gate of Chapter Seventeen: five fields, each with a reason, and no field whose purpose cannot be stated in one sentence.

Accuracy. Data must be correct and, where necessary, kept current. The engineering form is a correction path — a way for a person to say "this is wrong", and a mechanism to record the correction without destroying the history.

Storage limitation. Data must not be kept longer than necessary. The engineering form is a retention period per data category, enforced by a scheduled deletion. A system with no deletion job has an unbounded retention policy by default, which is a policy nobody chose.

Integrity and confidentiality. Appropriate security. The engineering form is least privilege on credentials, encryption in transit and at rest, and — the most common failure in small systems — no secret in the source repository.

Accountability. The controller must be able to demonstrate compliance. The engineering form is the provenance record of Chapter Six, plus written policies, plus a log of decisions. Accountability is the principle that turns the others from intentions into evidence.

20.3 The rights of the person

The regimes converge on a set of rights: to be informed, to access, to correct, to erasure, to object, and to portability. Each has an engineering consequence, and the con-

sequences are cheap if designed in and expensive if retro-fitted.

Right	Engineering consequence
To be informed	A notice at the point of collection, in plain language
To access	An export of everything held about a person, by identifier
To correct	An append-only correction event, not an in-place overwrite
To erasure	A deletion path that reaches every store, including backups, within a stated period
To object	A suppression list checked before any contact
To portability	A machine-readable export

The right to object is the one that a discovery system most needs, and it is the one that is easiest to get wrong: an objection must be recorded at the identity level and checked before a record is created, not at the campaign level and checked before a send.

20.4 Three legal questions before collection

A small team does not need a lawyer for every decision, and it does need an answer to three questions before it collects anything.

Whose law applies? If a merchant is in Zimbabwe, the buyer is in South Africa, the platform’s infrastructure is in a third country and the operator is in a fourth, four regimes have a plausible claim. The practical answer is to identify the regime of the data subject — the person the data is about — and to meet the strictest one in the set.

What is the basis, in one sentence? If the sentence cannot be written, the collection should not happen. “We collect the business’s name, address and telephone number from their public directory listing in order to contact them about a service relevant to their trade” is a sentence that can be defended or refused. “We collect data to improve our service” is not.

Who answers when it goes wrong? A named person, with the authority to stop a system. Systems without a named accountable person have no accountability, because accountability is a property of a person who can be asked.

This book is not legal advice and cannot be, because the answer to the first question depends on facts this book does not have. What it can do is insist that the questions are asked before a line of collection code is written, because after the fact the answer is a negotiation with a regulator rather than a design decision.

20.5 The obligations of a small operator, honestly

Large firms have compliance departments. A one-person operator has the same legal obligations and none of the staff, and the honest account of how to meet them is unglamorous.

Write the policies down, even at a page each. A retention policy that exists only in someone’s head is not a policy; it cannot be demonstrated, and accountability requires demonstration.

Automate the obligations that can be automated. Deletion jobs, suppression checks and export functions are code, and code runs without being remembered.

Keep a decision log. One line per decision, with a date and a reason. When a regulator or a customer asks why a system behaves as it does, the log is the answer.

Do not collect what you cannot protect. The minimisation principle is also a risk-control principle: every field you do not hold is a field that cannot leak.

Accept that some things will not be done. A one-person operator will not run a formal impact assessment for every feature, and pretending otherwise produces documents nobody reads. Prioritise the high-risk processing — special categories of data, large-scale monitoring, automated decisions with significant effects — and be plain about the rest.

20.6 Summary

The data-protection regimes of southern Africa and Europe share a common ancestry, so designing to the strictest applicable standard is the cheapest strategy for a multi-country operator. Seven principles each have an engineering form, and the rights of the data subject each have one too, most of them cheap to design in. Before collecting anything, answer whose law applies, what the basis is in one sentence, and who is accountable. A small operator meets the same obligations with fewer resources, and the honest response is written policies, automated obligations, a decision log, and restraint in what is collected.

Worked example — the retention schedule. Design one for a merchant platform. Answer: orders and statements — retained for the period required by tax law in each jurisdiction, then archived or deleted; prospect records — deleted if not contacted within 90 days, retained for 24

months after last contact, suppressed permanently on request; web logs — 30 days; support correspondence — 24 months. Each row has a period, a trigger and an enforcement mechanism, and the enforcement is a scheduled job. A schedule without a job is a wish.

Exercises. (recall) List the seven GDPR principles and the engineering form of each. (applied) Write a one-paragraph lawful-basis statement for a collection activity you could plausibly undertake. (critical) “Consent is the wrong basis for most business processing.” Discuss with reference to power asymmetry between a platform and a small merchant.

Further reading. GDPR (2016/679); POPIA (2013); Zimbabwe Cyber and Data Protection Act (2021); Kenya Data Protection Act (2019); Nigeria Data Protection Act (2023); [NITI Aayog], Unit 7.

CHAPTER 21

Teaching it: curriculum, sequence and assessment

Learning outcomes. By the end of this chapter you should be able to: compare the structures of the three source curricula; design a sequence for a stated cohort; and write an assessment that tests judgement rather than recall.

21.1 Three curricula, three designs

The three source documents are not the same kind of artefact, and comparing them is itself a lesson in instructional design.

COMCEC's introduction is a conceptual treatment for a policy-literate audience. It moves from definitions through history to the question of when AI should be used, and it carries the field's thought experiments, the four-definition grid, and a five-question decision formula [COMCEC]. Its unit of instruction is the idea: a reader finishes able to argue about AI rather than to build it. Its strength is that it makes the field's disagreements visible; its implicit assumption is an audience that will commission or evaluate rather than implement.

Microsoft's Module 1 is a vocational treatment. It opens with the need for AI, covers cloud storage and platforms, surveys applications in daily life, then works through APIs by family — vision, speech, language, search, decision — with assessments, labs, questions to ponder, further reading and a glossary [Microsoft]. Its unit of instruction is the

capability and its interface: a reader finishes able to select and call a service. Its strength is that everything it teaches is immediately usable; its implicit assumption is an audience with access to a cloud account.

NITI Aayog's Base Module is a school treatment, and the most ambitiously sequenced of the three. Seven units move from what intelligence is, through learning, data, visualisation, problem solving and algorithms, to languages, and finally to ethics [NITI Aayog]. Its unit of instruction is the skill, built in dependency order: you cannot visualise data you have not collected, and you cannot collect data without a question. Its strength is the sequence; its implicit assumption is a teacher with time.

A student of curriculum should notice that the three disagree about order without disagreeing about content. COMCEC puts judgement early; Microsoft puts capability early; NITI Aayog puts skill early. Each is right for its audience, and the choice of order is the design decision that makes a curriculum fit a cohort.

21.2 Sequencing for four cohorts

The practical question is which order fits which group.

Undergraduates in computing. NITI Aayog's order, then Microsoft's, then COMCEC's judgement material at the end. Skill first because they will build; capability second because they will be employed; judgement last because it is hardest to teach to people who have not yet seen a system fail.

Postgraduate students in any discipline. COMCEC first, then Parts Two and Three of this book in compressed form, then the case. The purpose is to produce people who

can evaluate systems, and the fastest route to evaluation is a taxonomy of failure.

Professional practitioners. Microsoft's order, then the case, then the ethics. They arrive with capability and leave needing judgement, and the case is the vehicle.

Non-specialists — managers, public servants, owners. COMCEC's order, plus chapters five, twelve, nineteen and twenty of this book. No code; the entire value is in the decision formula, the cost arithmetic, the prohibitions and the obligations.

21.3 The lab sequence

Appendix B gives eight labs. Their design principle is that a lab should produce an artefact the student would keep, and that no lab should require a paid service.

Labs one and two are paper: a data-quality audit and an evaluation plan. They teach the two skills that most determine whether a project succeeds, and they require no computer.

Lab three is a gate: given a hundred pages, write seven predicates, apply them by hand, and report the admission rate. The student discovers that the predicates interact and that the exclusion rules are the hard part.

Lab four is a normaliser: implement phone normalisation for three countries, with tests. It is the single most instructive programming exercise in the book, because the failure is silent and the tests can be written before the code.

Labs five and six are modelling: a logistic regression and a tree ensemble on a small tabular dataset, compared against a stated baseline, with a slice analysis.

Lab seven is a monitoring dashboard: five numbers, updated by script, read once a week.

Lab eight is a written judgement: a two-page assessment of a proposed system using the five-question test, the expected-cost calculation, and the ethical constraints. It is the closest thing in the book to the actual work.

21.4 Assessment that tests judgement

An examination that asks students to define precision measures recall. An examination that gives a confusion matrix and asks what to do measures judgement, and the marking scheme is where the teaching happens.

Three question forms work.

The threshold question. Here is a model, a cost structure and a capacity. Where do you set the threshold, and what would change your answer? Full marks require the calculation and the recognition that capacity, not the model, sets the operating point.

The missing-question question. Here is a proposal. Which of COMCEC's five questions has it not answered, and how would you find out? Full marks require identifying that the organisation question is usually the unasked one.

The silence question. Here is a system that reports success. What four things would you check? Full marks require the invariant — that zero output from non-empty input is an error.

Note what these have in common: each supplies a situation and asks for a decision, and each has a defensible range of answers. That is what professional assessment looks like in a field where the interesting problems are decisions rather than facts.

21.5 What cannot be taught in a semester

Two things, and honesty requires naming them.

Taste in problem selection. The ability to look at a messy situation and see which small problem is worth solving is learned by doing it repeatedly and being wrong. A course can supply the criteria — measurable, quickly solvable, important — and cannot supply the judgement.

The discipline to stop. Almost every student will, on their first real project, keep building past the point where the arithmetic said stop. The only reliable cure is having built something nobody used, and a course cannot arrange that on schedule.

What a course can do is install the two habits that make those lessons cheap: write the baseline before the model, and write the stopping rule before the results. Both fit on an index card.

21.6 Summary

The three curricula differ in ordering, not content: COMCEC teaches judgement, Microsoft teaches capability, NITI Aayog teaches skill. Sequence by cohort and by what the student must be able to do on leaving. Labs should produce artefacts and require no paid services. Assessment should supply situations and ask for decisions. Taste and the discipline to stop are learned in practice, and a course

can at least install the habits that make the learning cheap.

Worked example — sequencing a one-semester course for a mixed cohort. Twelve weeks, half computing students and half not. Answer: weeks 1–2 definitions and the four-question grid (all students); weeks 3–4 data quality and lifecycle, with the audit lab (paper, all students); week 5 evaluation and the confusion matrix; week 6 the arithmetic of being wrong, with the cost lab; weeks 7–8 learning and algorithms, split — computing students do the modelling lab, others do the gate lab; weeks 9–10 the case; week 11 ethics and law; week 12 the written judgement, assessed. The mixed cohort works because the shared language is decision-making, and the divergent labs meet the two groups' needs without splitting the class.

Exercises. (recall) State the organising principle of each of the three source curricula. (applied) Design a six-week module for a cohort you know, with a lab per week. (critical) "Assessment should reward the right decision, not the correct answer." Discuss.

Further reading. [COMCEC]; [Microsoft], Module 1, including its assessments and glossary; [NITI Aayog], all seven units; Biggs and Tang (2011) on constructive alignment.

CHAPTER 22

Conclusion: the discipline of the second question

22.1 What this book has argued

Five claims, in order.

Definitions are choices with consequences. Narrow and general, thinking and acting, human and rational — each distinction selects what a system is optimised for and which failures remain invisible. The first professional skill is to ask which definition a claim rests on.

Data determines what is learnable. Representation bounds performance, quality bounds representation, and leakage is the defect that makes a model perfect in testing and worthless in production. Most of the work, and most of the failures, are here.

Measurement is a moral act. The confusion matrix encodes a task's costs, the base rate determines whether an accurate system is useful, and the threshold allocates human attention. A team that reports accuracy without a baseline, a threshold and a base rate has not measured anything.

Deployment is where models become systems. Eight steps, each able to fail silently, and five failure classes that are invisible to the tests that would otherwise catch them. Monitoring is not optional, and architecture is the cheapest form of ethics.

Judgement is a practice. It is built from scenarios rehearsed, arithmetic done before building, and commit-

ments converted into constraints. It cannot be supplied by a course, and it can be started by one.

22.2 The second question

The title of this section is the book's practical residue, so it should be stated plainly.

The first question in any applied AI conversation is can it do this? The second question is what happens when it is wrong?

Almost all the failures in this book are failures to ask the second question. A gate that admits everything answers the first and not the second. A statement that cannot be recomputed answers the first and not the second. A copy that manufactures urgency answers the first and not the second. A model with 97% accuracy on a 4% base rate answers the first, and the second question reveals that the queue is useless.

The second question is not pessimism. It is the question that makes deployment possible, because a system whose failures are known and bounded can be trusted with real work, and a system whose failures are unknown cannot be trusted with anything. Optimism deploys nothing; a failure analysis deploys carefully.

22.3 What the case is for

Tapstall is in this book because it is small enough to see whole. A student can read its eleven thousand lines, its four hundred and nineteen assertions, its five thresholds and its seven predicates, and hold the entire system in mind — which is not possible with a system built by a thousand people.

What the case shows is that the discipline this book teaches is not a large-firm luxury. The quota ladder is thirty lines. The admission gate is seven predicates. The phone normaliser is a country table and a rule. The statement is arithmetic. The prohibitions are a checklist. Every one of these is available to a person with a laptop and a free account, and every one of them is a decision rather than a technology.

The case also shows the limit of that discipline. The system is good and it has no customers, and no amount of engineering rigour changes that. **Technique is necessary and not sufficient**; the sufficient condition involves a market, a price and a seller, and this book has almost nothing to say about those. A student who leaves believing that good engineering produces a good business has learned the wrong lesson from a good case.

22.4 Three things to do on Monday

The book ends with three actions, because a conclusion that only summarises is a conclusion that wastes the reader's remaining attention.

Write the baseline for whatever you are building. The trivial rule, its performance, and the number your system must beat. One line in a document. If you cannot write it, you do not yet know what problem you are solving.

Write the stopping rule. What result would cause you not to deploy? If the answer is "none", the project is not an experiment; it is a commitment, and commitments should be made deliberately rather than by default.

Name the person who answers when it is wrong. Not the team, not the vendor. A person, with the authority to

switch the system off. If no such person exists, the system has no accountability, and the first incident will be resolved by an argument rather than a decision.

Each of these takes under an hour and costs nothing. Together they are the difference between an organisation that uses AI and an organisation that is used by it.

22.5 Coda

The three curricula this book draws on were written to make a generation literate in a technology that arrived before its institutions did. That literacy is now a professional requirement rather than an aspiration, and its content is not what the marketing suggests. It is not the ability to train a model. It is the ability to say what a system is for, what it costs to be wrong, who is accountable, and what the system is forbidden to do.

Those four sentences are the whole book. Everything else — the definitions, the matrices, the ladder, the gate, the laws — is the means of answering them precisely enough to ship.

Appendix A · Glossary

Accuracy. The proportion of all predictions that are correct; misleading under class imbalance.

Acting rationally. The definition of AI that judges an agent by whether its actions are good given its information and objectives; the dominant framing in production systems.

Admission rule (gate). A set of predicates that a candidate record must satisfy before it enters a system, applied before the record exists rather than at the point of use.

Algorithm. A finite, unambiguous, terminating sequence of steps transforming an input into an output.

API. Application programming interface; a defined contract by which one program requests something from another.

Base rate. The proportion of positives in the population to which a system is applied; a property of the deployment, not the model.

Baseline. The trivial rule a model must beat; reported alongside every model result.

Bias (statistical). Systematic error in an estimate, as distinct from variance.

Bias–variance trade-off. The decomposition of expected error into systematic error from wrong assumptions, sensitivity to the training sample, and irreducible noise.

Calibration. The property that among items scored at probability p, approximately p are positive.

Chinese Room. Searle's thought experiment arguing that executing a program that produces correct answers does not constitute understanding.

Classification. Supervised prediction of a discrete label.

Clustering. Unsupervised grouping of records by similarity.

Confusion matrix. The two-by-two table of predicted against actual labels, from which precision, recall and specificity are computed.

Cross-validation. Repeated splitting of data into training and validation folds to estimate performance and its variability.

Custody (of funds). Holding customer money; excluded by design from the case's rail.

Data minimisation. Collecting only the data a stated purpose requires.

Decision tree. A model that recursively partitions data on feature thresholds.

Distribution shift. Change in the distribution of production inputs relative to training data; a primary cause of silent degradation.

Do-not-contact list. A record of persons or entities who have asked not to be contacted; checked before a record is created.

Entity extraction. Identifying specific details (dates, places, amounts) within an utterance or document.

Feature. A quantity computed from raw data and supplied to a model.

Feature engineering. Constructing representations that make the structure relevant to a task visible to a learner.

F1. The harmonic mean of precision and recall; assumes the two errors are equally costly.

Flat fee. A price independent of the customer's volume; contrasts with a percentage fee.

Gradient descent. Iterative parameter update in the direction that reduces a loss function.

Intent classification. Deciding what a user wants from an utterance; paired with entity extraction in conversational systems.

Leakage. Presence in training features of information unavailable at prediction time, or of a consequence of the outcome.

Narrow AI. A system effective only in the domain it was trained for.

General AI. A hypothetical system with human-like general competence; does not exist.

Overfitting. Fitting noise in the training data, producing good training and poor test performance.

Precision. Of the items flagged, the proportion that are correct.

Provenance. The record of where a value came from, when, and by what process.

Pseudocode. Language-independent description of a procedure, intended to be read by people.

Recall (sensitivity). Of the items that should be flagged, the proportion that are.

Recomputability. The property that every figure in a document can be re-derived from underlying records by a stated rule.

Regularisation. A penalty on model complexity added to the loss to reduce variance.

Reserve ladder. A series of descending thresholds, each admitting a class of operations, used to protect critical operations under resource pressure.

Reinforcement learning. Learning a policy through actions, rewards and interaction with an environment.

Specificity. Of the genuinely negative items, the proportion correctly left alone.

Starvation freedom. The property that a class of operations cannot be denied service by the consumption of other classes.

Supervised learning. Learning a mapping from labelled input–output pairs.

Test set. Data used once, at the end, to estimate performance on unseen data.

Threshold. The score above which a record is flagged; determines volume of human work and is therefore a governance decision.

Turing test. Turing's 1950 operational test of indistinguishability between machine and human in text conversation.

Unsupervised learning. Finding structure in unlabelled data.

Validation set. Data used to choose hyperparameters and compare models.

Appendix B · Eight labs

Each lab produces an artefact. None requires a paid service. Labs 1, 2 and 8 require no code.

Lab 1 — Data-quality audit. Take a dataset of at least fifty records. Produce a scorecard with columns: dimension, field, failure rate, consequence. Include leakage and representativeness. Artefact: one-page scorecard and a recommendation.

Lab 2 — Evaluation plan. For a task of your choice, write the seven-part plan of §11.5: decision, unit, metric and threshold, baseline, split, slices, stopping rule. Artefact: one page, written before any modelling.

Lab 3 — The gate. Take one hundred pages from a public directory. Write seven predicates, one of which must be an exclusion you would defend to the person excluded. Apply them by hand. Report the admission rate and the distribution of rejection reasons. Artefact: the predicate list and a table of rejections. Question to answer in writing: what does your admission rate tell you about the source?

Lab 4 — The normaliser. Implement phone normalisation for three countries, including the trunk-prefix case described in §17.4. Write the tests first: at least twenty cases, including bracketed zeros, spaces, local-only numbers, extensions, and non-numbers. Artefact: code and passing tests.

Lab 5 — Baseline and model. On a small tabular dataset, compute a trivial baseline, then fit logistic regression and a random forest with five-fold cross-validation. Report

mean and spread for each. Artefact: a table of results and one paragraph explaining the difference.

Lab 6 — Cost and threshold. Using the confusion matrix from Lab 5, compute expected cost under two cost structures, and select a threshold for each. Report the change in volume of flagged items. Artefact: two thresholds, two volumes, one recommendation.

Lab 7 — Monitoring. Instrument a small script to record five daily numbers: volume, rejection rate, invariant failures, outcome rate, cost per unit. Write a one-page monthly review template. Artefact: the dashboard and the template.

Lab 8 — Written judgement. For a proposed system of your choice, produce a two-page assessment: the five-question test of §4.3, the expected-cost calculation of §12.1, the ethical constraints of §19.4, and a recommendation with a stopping rule. Artefact: two pages, no code, assessed on the quality of the decision.

Appendix C · Mapping to the three source curricula

The three tables below indicate which chapters of this book extend which sections of each source document, so that an instructor teaching from any one of them can find the corresponding applied treatment, worked example and exercises.

C.1 COMCEC — Introduction to Artificial Intelligence and Fundamental Concepts (Kurt, 2025)

This book	COMCEC section extended
1 What intelligence is	Definitions; Turing test; Chinese Room; narrow vs general; Cahit Arf
2 Four definitions	The thinking/acting × human/rational grid; the eight quoted definitions
3 History	Dartmouth 1956; the laboratories; Hopfield and Hinton; Hassabis and Jumper
4 What AI can do	Capabilities and limits; “not a superhero”; the five-question formula
5 The small firm	The five-question formula, especially the organisation question
6 Data	Data and its quality
7 Lifecycle	Collection, operations, modelling, deployment, results, validation
8 Features	Feature engineering; the Ng quotation
10 Learning	Machine learning within the lifecycle
11 Measurement	Evaluating models
12 Being wrong	Why data quality and use-case definition precede technique
13 Production	Deployment and results collection
19 Ethics	The limits of AI
21 Teaching	The document as a whole
22 Conclusion	The document as a whole

C.2 Microsoft — Fundamentals of Artificial Intelligence, Module 1 (2020)

This book	Microsoft section extended
1 What intelligence is	McCarthy's definition; the faculties of intelligence
4 What AI can do	The need for AI; applications in daily life
5 The small firm	The need for AI
6 Data	Public databases and datasets
11 Measurement	The module's assessments and labs
13 Production	Cloud platforms; the five API families
15 Languages, cloud, APIs	Cloud categories; the named services; conversational AI
16 The rail	Cloud storage and its cost structure
17 The gate	Conversational AI: intent classification and entity extraction
20 Law	Data and the cloud
21 Teaching	The module as a whole, including assessments and glossary
22 Conclusion	The module as a whole

C.3 NITI Aayog — Let's Learn Artificial Intelligence, Base Module (2020)

This book	NITI Aayog unit extended
1 What intelligence is	Unit 1: human intelligence; the intelligent agent
4 What AI can do	Unit 1: components of AI; the future of AI
5 The small firm	Unit 1: why we need AI
6 Data	Unit 3: Data
7 Lifecycle	Unit 3: Data
9 Visualisation	Unit 4: Data Visualisation
10 Learning	Unit 2: Learning
14 Algorithms	Unit 5: Problem Solving and Decision Making; Unit 6: pseudocode
15 Languages	Unit 6: Scratch and Python
19 Ethics	Unit 7: Ethics
20 Law	Unit 7: Ethics — consent, anonymity, refusal
21 Teaching	All seven units
22 Conclusion	All seven units

An instructor can also read these tables in reverse: locate the section being taught in the source document, and read the corresponding chapter of this book for the applied treatment, the worked example and the exercises.

Appendix D · References and sources

The three source curricula

Kurt, M. P. (2025). Introduction to Artificial Intelligence and Fundamental Concepts. Ankara: Standing Committee for Economic and Commercial Cooperation of the Organisation of Islamic Cooperation (COMCEC). Cited in this book as [COMCEC].

Microsoft (2020). Fundamentals of Artificial Intelligence: Student Guide, Module 1. Microsoft Corporation. Cited in this book as [Microsoft].

NITI Aayog (2020). Let's Learn Artificial Intelligence: Base Module. New Delhi: Atal Innovation Mission, NITI Aayog, with NASSCOM. Cited in this book as [NITI Aayog].

Works quoted or relied on in the text

Anscombe, F. J. (1973). Graphs in statistical analysis. The American Statistician, 27(1), 17–21.

Biggs, J., and Tang, C. (2011). Teaching for Quality Learning at University (4th ed.). Maidenhead: Open University Press.

Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). The ML test score: a rubric for ML production readiness and technical debt reduction. IEEE International Conference on Big Data.

Cairo, A. (2016). The Truthful Art: Data, Charts, and Maps for Communication. San Francisco: New Riders.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms (4th ed.). Cambridge, MA: MIT Press.

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

Floridi, L., and Cowls, J. (2019). A unified framework of five principles for AI in society. Harvard Data Science Review, 1(1).

Gigerenzer, G., Gaissmaier, W., Kurz-Milcke, E., Schwartz, L. M., and Woloshin, S. (2007). Helping doctors and patients make sense of health statistics. Psychological Science in the Public Interest, 8(2), 53–96.

Hastie, T., Tibshirani, R., and Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). New York: Springer.

Kahneman, D., and Tversky, A. (1973). On the psychology of prediction. Psychological Review, 80(4), 237–251.

Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms (3rd ed.). Reading, MA: Addison-Wesley.

Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25.

McCarthy, J., Minsky, M. L., Rochester, N., and Shannon, C. E. (1955). A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence.

Niculescu-Mizil, A., and Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of the 22nd International Conference on Machine Learning.

Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (2nd ed.). Raleigh, NC: Pragmatic Bookshelf.

Redman, T. C. (2008). Data Driven: Profiting from Your Most Important Business Asset. Boston: Harvard Business Press.

Russell, S., and Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Harlow: Pearson.

Saito, T., and Rehmsmeier, M. (2015). The precision–recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3).

Sarasvathy, S. D. (2008). Effectuation: Elements of Entrepreneurial Expertise. Cheltenham: Edward Elgar.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. Advances in Neural Information Processing Systems, 28.

Searle, J. R. (1980). Minds, brains, and programs. Behavioral and Brain Sciences, 3(3), 417–424.

Simon, H. A. (1955). A behavioral model of rational choice. The Quarterly Journal of Economics, 69(1), 99–118.

Sutton, R. S., and Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). Cambridge, MA: MIT Press.

Tufte, E. R. (2001). The Visual Display of Quantitative Information (2nd ed.). Cheshire, CT: Graphics Press.

Turing, A. M. (1950). Computing machinery and intelligence. Mind, 59(236), 433–460.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

Zheng, A., and Casari, A. (2018). Feature Engineering for Machine Learning. Sebastopol, CA: O’Reilly.

Legal instruments

Council of Europe / European Union. Regulation (EU) 2016/679 (General Data Protection Regulation).

Kenya. Data Protection Act, No. 24 of 2019.

Nigeria. Nigeria Data Protection Act, 2023.

Republic of South Africa. Protection of Personal Information Act, No. 4 of 2013.

Republic of Zimbabwe. Cyber and Data Protection Act [Chapter 12:07], 2021.

The author’s own work

Garikai, C. (2026). Order Without Custody: A Settlement Rail for Merchants the Platforms Forgot. Research paper, Harare.

Garikai, C. (2026). The Ledger and the Shopkeeper: Building Commerce for the Markets the Platforms Forgot. Harare: Jacaranda Labs Press.

A note on citation practice

The three source curricula are cited by author or institution and year, and specific ideas are attributed in the text at the point of use. Direct quotations from those documents are marked and attributed; paraphrases are attributed without quotation marks. No page numbers are given for the source curricula, because the document versions used are digital and pagination varies between copies — an instructor wanting to locate a passage should use the section names given in Appendix C.

References to production figures for the Tapstall case refer to release 1.9.1 of the system and to the figures current at the time of writing.

Colophon

Intelligence, Data and Judgement: An applied introduction to artificial intelligence, with a production case

First edition, October 2026.

Written by Courage Garikai. Published by Jacaranda Labs Press, Harare.

Typeset at 10.5 point on 15.5 point leading on a 6 × 9 inch page, with running heads, a contents list generated from the document structure, and part openers on their own pages. The print edition was produced from a single Markdown source via HTML and CSS, without a word processor, so that text and pagination remain reproducible.

The book is released under a Creative Commons Attribution 4.0 International licence. You may copy, redistribute, remix and build upon it, including commercially, provided you attribute the author and indicate changes. The three source curricula are the property of their respective publishers and are used here with attribution under their own terms; a reader redistributing this book should verify those terms before redistributing substantial portions of the source material.

The case described in Part Four is a live commercial system. Its figures, thresholds, predicates and failure accounts are reported as accurately as the author can make them, and the limitations stated in §18.5 — in particular the absence of paying customers — are part of the account rather than a disclaimer appended to it.